

```
// ===== recycle-admin/server/server.js =====
```

```
/**
 * 电子产品回收综合服务平台 - 独立后台服务
 *
 * 与维修后台管理系统完全独立部署（同域名、不同端口，默认 3005）。
 * 生产模式直接托管 web/dist 静态资源，单端口即可对外提供完整后台。
 */
const express = require('express');
const cors = require('cors');
const path = require('path');
const fs = require('fs');
require('dotenv').config();

const db = require('./database');
const { authenticate } = require('./middleware/auth');
const authRoutes = require('./routes/authRoutes');
const orderRoutes = require('./routes/orderRoutes');
const catalogRoutes = require('./routes/catalogRoutes');
const { router: platformRoutes } = require('./routes/platformRoutes');
const linkRoutes = require('./routes/linkRoutes');
const statsRoutes = require('./routes/statsRoutes');
const settingRoutes = require('./routes/settingRoutes');
const logRoutes = require('./routes/logRoutes');

const app = express();
app.use(cors());
app.use(express.json({ limit: '5mb' }));
app.use(express.urlencoded({ extended: true }));

// 请求日志（简要）
app.use((req, res, next) => {
  if (req.path.startsWith('/api')) {
    console.log(`${new Date().toLocaleString('zh-CN')} ${req.method} ${req.path}`);
  }
  next();
});

// 业务路由（统一前缀 /api）
app.use('/api/auth', authRoutes);
app.use('/api/orders', authenticate, orderRoutes);
app.use('/api/catalog', authenticate, catalogRoutes);
app.use('/api/platforms', authenticate, platformRoutes);
app.use('/api/links', authenticate, linkRoutes);
app.use('/api/stats', authenticate, statsRoutes);
app.use('/api/system', settingRoutes);
app.use('/api/logs', logRoutes);

// 健康检查
app.get('/health', async (req, res) => {
  try {
    await db.query('SELECT 1');
```

```
res.json({ status: 'OK', service: 'recycle-admin', time: new Date().toISOString() });
} catch (e) {
  res.status(500).json({ status: 'ERROR', message: e.message });
}
});

// 统一错误处理
app.use((err, req, res, next) => {
  console.error('[Error]', err);
  const status = err.status || 500;
  res.status(status).json({ success: false, message: status === 500 ? '服务器错误' : err.message });
});

// 生产模式：托管前端构建产物（web/dist），支持前端路由 history 模式
const distDir = path.join(__dirname, '..', 'web', 'dist');
if (fs.existsSync(distDir)) {
  app.use(express.static(distDir));
  app.get(/^(!\Vapi\|Vhealth).*/i, (req, res) => {
    res.sendFile(path.join(distDir, 'index.html'));
  });
}

app.use((req, res) => {
  res.status(404).json({ success: false, message: '接口不存在' });
});

const PORT = parseInt(process.env.PORT) || 3005;

db.init()
  .then(() => {
    app.listen(PORT, '0.0.0.0', () => {
      console.log('=====');
      console.log(' 电子产品回收综合服务平台 - 后台服务已启动');
      console.log(' API 地址: http://localhost:${PORT}/api');
      console.log(' 管理后台: http://localhost:${PORT} (已托管 web/dist 时)');
      console.log(' 开发前端: http://localhost:5175 (web 目录 npm run dev)');
      console.log(' 默认账号: admin / admin123 (首次登录后请修改)');
      console.log('=====');
    });
  })
  .catch((err) => {
    console.error('服务启动失败（数据库不可用）:', err.message);
    process.exit(1);
  });

// ===== recycle-admin/server/database.js =====

/**
 * 回收综合服务平台 - 数据库连接与表结构管理
 *
 * 独立于维修后台管理系统运行，但共用同一个 MySQL 实例/库：
 * - 回收订单读取业务库现有 orders 表（order_type='recycle'），不重复建表
 * - 配价库/平台/链接/配置等为本系统专属表（recycle_ 前缀），首次启动自动建表并灌入种子数据
```

```
*/
const mysql = require('mysql2/promise');
const bcrypt = require('bcryptjs');
const fs = require('fs');
const path = require('path');
require('dotenv').config();

class Database {
  constructor() {
    this.pool = null;
    this.readyPromise = null;
  }

  async init() {
    if (this.readyPromise) return this.readyPromise;
    this.readyPromise = this._init();
    return this.readyPromise;
  }

  async _init() {
    this.pool = mysql.createPool({
      host: process.env.DB_HOST || 'localhost',
      port: parseInt(process.env.DB_PORT) || 3306,
      user: process.env.DB_USER || 'root',
      password: process.env.DB_PASSWORD || '',
      database: process.env.DB_NAME || 'repair',
      connectionLimit: parseInt(process.env.DB_CONNECTION_LIMIT) || 10,
      connectTimeout: 10000,
      charset: 'utf8mb4',
      timezone: '+08:00',
      dateStrings: true,
      waitForConnections: true,
      queueLimit: 0
    });

    this.pool.on('error', (err) => {
      console.error('[DB] 连接池错误:', err.code || err.message);
    });

    const conn = await this.pool.getConnection();
    try {
      await conn.ping();
      await this.ensureTables(conn);
      await this.seedDefaults(conn);
    } finally {
      conn.release();
    }
    console.log('[DB] 已连接 ${process.env.DB_HOST}:${process.env.DB_PORT}/${process.env.DB_NAME}');
  }

  async query(sql, params = []) {
    await this.init();
  }
}
```

```
const [rows] = await this.pool.query(sql, params);
return rows;
}

/**
 * 业务库 orders 表结构随小程序后端迭代，可选列在此登记，
 * 查询订单前调用，避免本地/服务器结构差异导致 500。
 */
async orderColumns() {
  if (this._orderColumns) return this._orderColumns;
  const rows = await this.query(
    `SELECT COLUMN_NAME AS name FROM INFORMATION_SCHEMA.COLUMNS
     WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'orders'`
  );
  this._orderColumns = new Set(rows.map((r) => r.name));
  return this._orderColumns;
}

/**
 * user_addresses 表同样存在环境间结构差异（部分环境无 detail 列），
 * 订单详情联查前调用，按需拼接字段。
 */
async addressColumns() {
  if (this._addressColumns) return this._addressColumns;
  const rows = await this.query(
    `SELECT COLUMN_NAME AS name FROM INFORMATION_SCHEMA.COLUMNS
     WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'user_addresses'`
  );
  this._addressColumns = new Set(rows.map((r) => r.name));
  return this._addressColumns;
}

async ensureTables(conn) {
  const ddl = [
    // 管理员（本系统独立账号体系）
    `CREATE TABLE IF NOT EXISTS recycle_admins (
      id INT PRIMARY KEY AUTO_INCREMENT,
      username VARCHAR(64) NOT NULL COMMENT '登录账号',
      password_hash VARCHAR(128) NOT NULL COMMENT 'bcrypt密码',
      name VARCHAR(64) NOT NULL DEFAULT '' COMMENT '姓名',
      role ENUM('super','admin','viewer') NOT NULL DEFAULT 'admin' COMMENT '角色: super超管/admin管理员/viewer只读',
      status TINYINT(1) NOT NULL DEFAULT 1 COMMENT '状态: 1启用 0禁用',
      last_login_at DATETIME NULL,
      created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
      updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
      UNIQUE KEY uniq_username (username)
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='回收后台管理员'`,

    // 设备配价库：分类
    `CREATE TABLE IF NOT EXISTS recycle_categories (
      id INT PRIMARY KEY AUTO_INCREMENT,
      code VARCHAR(32) NOT NULL COMMENT '分类编码',
```

```
name VARCHAR(64) NOT NULL COMMENT '分类名称',
icon VARCHAR(16) DEFAULT '' COMMENT '图标(emoji)',
color VARCHAR(16) DEFAULT '#5B9E8A' COMMENT '主题色',
sort_order INT DEFAULT 0 COMMENT '排序',
status TINYINT(1) NOT NULL DEFAULT 1 COMMENT '状态: 1上架 0下架',
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
UNIQUE KEY uniq_code (code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='回收设备分类',

// 设备配价库: 品牌
`CREATE TABLE IF NOT EXISTS recycle_brands (
  id INT PRIMARY KEY AUTO_INCREMENT,
  category_id INT NOT NULL COMMENT '所属分类ID',
  name VARCHAR(64) NOT NULL COMMENT '品牌名称',
  logo_text VARCHAR(16) DEFAULT '' COMMENT '品牌字标',
  logo_color VARCHAR(16) DEFAULT '#666666' COMMENT '品牌色',
  sort_order INT DEFAULT 0,
  status TINYINT(1) NOT NULL DEFAULT 1,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_category (category_id),
  CONSTRAINT fk_recycle_brand_cat FOREIGN KEY (category_id) REFERENCES recycle_categories(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='回收设备品牌',

// 设备配价库: 型号 (配价核心表)
`CREATE TABLE IF NOT EXISTS recycle_models (
  id INT PRIMARY KEY AUTO_INCREMENT,
  brand_id INT NOT NULL COMMENT '所属品牌ID',
  name VARCHAR(128) NOT NULL COMMENT '型号名称',
  specs VARCHAR(255) DEFAULT '' COMMENT '规格说明',
  base_price DECIMAL(10,2) NOT NULL DEFAULT 0 COMMENT '回收基准价(最高回收价)',
  market_price DECIMAL(10,2) NULL COMMENT '二手市场参考价',
  hot TINYINT(1) NOT NULL DEFAULT 0 COMMENT '是否热门机型',
  status TINYINT(1) NOT NULL DEFAULT 1 COMMENT '状态: 1上架 0下架',
  sort_order INT DEFAULT 0,
  updated_by INT NULL COMMENT '最后调价管理员',
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_brand (brand_id),
  INDEX idx_status_hot (status, hot),
  CONSTRAINT fk_recycle_model_brand FOREIGN KEY (brand_id) REFERENCES recycle_brands(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='回收设备型号配价',

// 调价记录 (配价审计)
`CREATE TABLE IF NOT EXISTS recycle_price_logs (
  id INT PRIMARY KEY AUTO_INCREMENT,
  model_id INT NOT NULL,
  model_name VARCHAR(128) DEFAULT '',
  old_price DECIMAL(10,2) NULL,
  new_price DECIMAL(10,2) NOT NULL,
  reason VARCHAR(255) DEFAULT '',
```

```
admin_id INT NULL,
admin_name VARCHAR(64) DEFAULT "",
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
INDEX idx_model (model_id),
INDEX idx_created (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='配价调整记录',
```

// 回收平台信息 (爱回收/转转等回收网站与平台档案)

```
`CREATE TABLE IF NOT EXISTS recycle_platforms (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(64) NOT NULL COMMENT '平台名称',
  url VARCHAR(255) NOT NULL DEFAULT "" COMMENT '平台网址',
  type ENUM('recycle','procurement','compare') NOT NULL DEFAULT 'recycle' COMMENT '类型: recycle回收平台/procurem...',
  logo_text VARCHAR(16) DEFAULT "",
  logo_color VARCHAR(16) DEFAULT '#5B9E8A',
  description VARCHAR(500) DEFAULT "" COMMENT '平台说明',
  service_mode VARCHAR(128) DEFAULT "" COMMENT '服务方式: 上门/邮寄/到店',
  settlement VARCHAR(128) DEFAULT "" COMMENT '结算方式',
  commission_desc VARCHAR(128) DEFAULT "" COMMENT '佣金/费率说明',
  contact VARCHAR(128) DEFAULT "" COMMENT '联系方式',
  click_count INT NOT NULL DEFAULT 0 COMMENT '跳转点击量',
  status TINYINT(1) NOT NULL DEFAULT 1,
  sort_order INT DEFAULT 0,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_type (type),
  INDEX idx_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='回收/采购平台信息',
```

// 采购链接 (按设备分类/机型存储的外部采购与比价链接, 支持一键跳转)

```
`CREATE TABLE IF NOT EXISTS recycle_procurement_links (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(128) NOT NULL COMMENT '链接名称',
  url VARCHAR(500) NOT NULL COMMENT '链接地址',
  category_id INT NULL COMMENT '适用设备分类(空=通用)',
  platform_id INT NULL COMMENT '关联平台',
  model_keyword VARCHAR(128) DEFAULT "" COMMENT '适用型号关键词',
  price_range VARCHAR(64) DEFAULT "" COMMENT '参考价格区间',
  notes VARCHAR(500) DEFAULT "" COMMENT '备注',
  click_count INT NOT NULL DEFAULT 0 COMMENT '跳转点击量',
  last_click_at DATETIME NULL,
  status TINYINT(1) NOT NULL DEFAULT 1,
  sort_order INT DEFAULT 0,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_category (category_id),
  INDEX idx_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='采购网站平台链接',
```

// 链接/平台跳转日志

```
`CREATE TABLE IF NOT EXISTS recycle_click_logs (
  id INT PRIMARY KEY AUTO_INCREMENT,
```

```
target_type ENUM('platform','link') NOT NULL,
target_id INT NOT NULL,
target_name VARCHAR(128) DEFAULT "",
source VARCHAR(32) DEFAULT 'admin' COMMENT '来源: admin后台/mini小程序',
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
INDEX idx_target (target_type, target_id),
INDEX idx_created (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='跳转点击日志',

// 估价配置 (成色/屏幕/功能等系数项)
`CREATE TABLE IF NOT EXISTS recycle_condition_rates (
  id INT PRIMARY KEY AUTO_INCREMENT,
  factor_key VARCHAR(32) NOT NULL COMMENT '因子: condition/screen/function/version/accessories/repair_history',
  factor_name VARCHAR(32) NOT NULL COMMENT '因子名称',
  label VARCHAR(64) NOT NULL COMMENT '选项文案',
  value VARCHAR(32) NOT NULL COMMENT '选项值',
  rate DECIMAL(5,2) NOT NULL DEFAULT 1.00 COMMENT '价格系数',
  sort_order INT DEFAULT 0,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_factor (factor_key)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='估价系数配置',

// 系统参数
`CREATE TABLE IF NOT EXISTS recycle_settings (
  id INT PRIMARY KEY AUTO_INCREMENT,
  config_key VARCHAR(64) NOT NULL,
  config_value TEXT,
  description VARCHAR(255) DEFAULT "",
  updated_by INT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_key (config_key)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='回收系统参数',

// 操作日志
`CREATE TABLE IF NOT EXISTS recycle_operation_logs (
  id INT PRIMARY KEY AUTO_INCREMENT,
  admin_id INT NULL,
  admin_name VARCHAR(64) DEFAULT "",
  module VARCHAR(32) NOT NULL COMMENT '模块: order/catalog/platform/link/setting/auth',
  action VARCHAR(64) NOT NULL COMMENT '操作',
  detail VARCHAR(1000) DEFAULT "",
  ip VARCHAR(64) DEFAULT "",
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_module (module),
  INDEX idx_created (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='后台操作日志'
];

for (const sql of ddl) {
  await conn.query(sql);
}
```

```
}
console.log('[DB] recycle_* 表结构已就绪');
}

/**
 * 首次启动种子数据（均为幂等：仅在对应表为空时写入）
 */
async seedDefaults(conn) {
  // 1. 默认管理员
  const [adminRows] = await conn.query('SELECT COUNT(*) AS c FROM recycle_admins');
  if (Number(adminRows[0].c) === 0) {
    const hash = bcrypt.hashSync(
      process.env.DEFAULT_ADMIN_PASSWORD || 'admin123',
      10
    );
    await conn.query(
      `INSERT INTO recycle_admins (username, password_hash, name, role)
      VALUES (?, ?, ?, 'super')`,
      [process.env.DEFAULT_ADMIN_USERNAME || 'admin', hash, process.env.DEFAULT_ADMIN_NAME || '系统管理员']
    );
    console.log('[DB] 已创建默认管理员');
  }

  // 2. 配价目录（分类→品牌→型号）
  const [catRows] = await conn.query('SELECT COUNT(*) AS c FROM recycle_categories');
  if (Number(catRows[0].c) === 0) {
    const catalogFile = path.join(__dirname, 'data', 'catalog.json');
    if (fs.existsSync(catalogFile)) {
      const catalog = JSON.parse(fs.readFileSync(catalogFile, 'utf8'));
      for (const cat of catalog.categories || []) {
        const [catRes] = await conn.query(
          `INSERT INTO recycle_categories (code, name, icon, color, sort_order) VALUES (?, ?, ?, ?, ?)`,
          [cat.code, cat.name, cat.icon, cat.color, cat.sortOrder]
        );
        for (const brand of cat.brands || []) {
          const [brandRes] = await conn.query(
            `INSERT INTO recycle_brands (category_id, name, logo_text, logo_color, sort_order) VALUES (?, ?, ?, ?, ?)`,
            [catRes.insertId, brand.name, brand.logoText, brand.logoColor, brand.sortOrder]
          );
          for (const model of brand.models || []) {
            await conn.query(
              `INSERT INTO recycle_models (brand_id, name, specs, base_price, market_price, hot, sort_order)
              VALUES (?, ?, ?, ?, ?, ?, ?)`,
              [brandRes.insertId, model.name, model.specs, model.basePrice, model.marketPrice, model.hot, model.sortOrder]
            );
          }
        }
      }
    }
  }

  const [cnt] = await conn.query(
    `SELECT (SELECT COUNT(*) FROM recycle_categories) AS cats,
    (SELECT COUNT(*) FROM recycle_brands) AS brands,
    (SELECT COUNT(*) FROM recycle_models) AS models`
  );
}
```

```
);  
console.log('[DB] 已导入配价种子: ${cnt[0].cats}分类/${cnt[0].brands}品牌/${cnt[0].models}型号');  
}  
}
```

// 3. 估价系数

```
const [rateRows] = await conn.query('SELECT COUNT(*) AS c FROM recycle_condition_rates');  
if (Number(rateRows[0].c) === 0) {  
  const catalogFile = path.join(__dirname, 'data', 'catalog.json');  
  const factorNames = {  
    condition: '设备成色', screen: '屏幕状况', function: '功能状况',  
    version: '设备版本', accessories: '配件状况', 'repair-history': '维修史'  
  };  
  if (fs.existsSync(catalogFile)) {  
    const catalog = JSON.parse(fs.readFileSync(catalogFile, 'utf8'));  
    let idx = 0;  
    for (const q of catalog.guideQuestions || []) {  
      if (!q.options) continue;  
      for (const opt of q.options) {  
        idx += 1;  
        await conn.query(  
          `INSERT INTO recycle_condition_rates (factor_key, factor_name, label, value, rate, sort_order)  
          VALUES (?, ?, ?, ?, ?)`,  
          [q.id, factorNames[q.id] || q.id, opt.label, opt.value, opt.rate || 1, idx]  
        );  
      }  
    }  
    console.log('[DB] 已导入估价系数配置');  
  }  
}
```

// 4. 系统参数默认值

```
const defaultSettings = [  
  ['base_factor', '0.9', '二手回收基准系数（即便全新也按此折扣收）'],  
  ['ai_evaluate_enabled', '1', '小程序端 AI 估价开关: 1开 0关'],  
  ['price_valid_days', '3', '配价有效期(天), 超期需重新询价'],  
  ['service_phone', '400-888-8888', '回收服务电话'],  
  ['recycle_notice', '回收前请备份数据并退出账号；工程师验机后确定最终价格。', '回收须知'],  
  ['max_price_adjust_percent', '20', '单次调价幅度上限(%), 超过需二次确认'],  
  // 环保贡献换算系数（行业估算口径，可在系统设置中调整）  
  ['eco_co2_per_device', '25', '环保换算：每台设备回收减少碳排放(kg CO2e)'],  
  ['eco_tree_co2_year', '18', '环保换算：一棵树年吸收二氧化碳(kg)，用于等效植树'],  
  ['eco_energy_per_device', '8.7', '环保换算：每台设备回收节省电能(kWh)']  
];  
for (const [key, value, desc] of defaultSettings) {  
  await conn.query(  
    'INSERT IGNORE INTO recycle_settings (config_key, config_value, description) VALUES (?, ?, ?)',  
    [key, value, desc]  
  );  
}
```

// 5. 示例回收平台（可编辑/删除）

```

const [pfRows] = await conn.query('SELECT COUNT(*) AS c FROM recycle_platforms');
if (Number(pfRows[0].c) === 0) {
  const samples = [
    ['爱回收', 'https://www.aihuishou.com', 'recycle', '爱', '#00C29A', '京东旗下二手回收平台, 支持上门/邮寄/到店', '上门/邮寄/到...'],
    ['转转回收', 'https://www.zhuanzhuan.com', 'recycle', '转', '#FF5A5F', '二手交易平台, 可自主挂卖或官方回收', '邮寄/到店', ...],
    ['闲鱼', 'https://www.goofish.com', 'compare', '闲', '#FFAA00', '阿里旗下闲置交易, 适合自主定价出售比价', '线上交易', '买家确认后...'],
    ['回收宝', 'https://www.huishoubao.net', 'recycle', '宝', '#4A90D9', '二手数码回收平台, 与多家门店合作', '上门/邮寄', '验机后24h打...']
  ];
  let i = 0;
  for (const s of samples) {
    i += 1;
    await conn.query(
      `INSERT INTO recycle_platforms (name, url, type, logo_text, logo_color, description, service_mode, settlement, commission_d...
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?),
      [...s, i]
    );
  }
  console.log('[DB] 已写入示例回收平台');
}
}

module.exports = new Database();

// ===== recycle-admin/server/middleware/auth.js =====

/**
 * 回收后台 JWT 认证与角色控制
 */
const jwt = require('jsonwebtoken');
const db = require('../database');

const JWT_SECRET = () => process.env.JWT_SECRET || 'recycle-admin-jwt-secret-change-in-production';

function signToken(admin) {
  return jwt.sign(
    { adminId: admin.id, username: admin.username, role: admin.role },
    JWT_SECRET(),
    { expiresIn: process.env.JWT_EXPIRES_IN || '12h' }
  );
}

/** 解析并校验 token (支持 Authorization 头与 ?token= 查询参数, 后者用于文件导出等新窗口场景) */
async function authenticate(req, res, next) {
  try {
    const header = req.headers['authorization'] || '';
    const token = header.startsWith('Bearer ') ? header.slice(7) : (req.query.token || '');
    if (!token) {
      return res.status(401).json({ success: false, message: '未登录' });
    }
    const decoded = jwt.verify(token, JWT_SECRET());
    const rows = await db.query(
      'SELECT id, username, name, role, status, last_login_at FROM recycle_admins WHERE id = ?',

```

```
[decoded.adminId]
);
if (rows.length === 0) {
  return res.status(401).json({ success: false, message: '账号不存在' });
}
const admin = rows[0];
if (admin.status !== 1) {
  return res.status(403).json({ success: false, message: '账号已禁用' });
}
req.admin = admin;
next();
} catch (err) {
  return res.status(401).json({ success: false, message: '登录已过期, 请重新登录' });
}
}

/** 写操作需要 admin 或 super 角色 (viewer 只读) */
function requireWrite(req, res, next) {
  if (!req.admin) return res.status(401).json({ success: false, message: '未登录' });
  if (req.admin.role === 'viewer') {
    return res.status(403).json({ success: false, message: '只读账号无权执行该操作' });
  }
  next();
}

/** super 专属 (管理员管理、危险操作) */
function requireSuper(req, res, next) {
  if (!req.admin) return res.status(401).json({ success: false, message: '未登录' });
  if (req.admin.role !== 'super') {
    return res.status(403).json({ success: false, message: '需要超级管理员权限' });
  }
  next();
}

module.exports = { signToken, authenticate, requireWrite, requireSuper };

// ===== recycle-admin/server/utils/oplog.js =====

/**
 * 操作日志记录
 */
const db = require('../database');

async function writeOpLog(req, module, action, detail) {
  try {
    await db.query(
      `INSERT INTO recycle_operation_logs (admin_id, admin_name, module, action, detail, ip)
      VALUES (?,?,?,?,?,?)`,
      [
        req.admin ? req.admin.id : null,
        req.admin ? (req.admin.name || req.admin.username) : '',
        module,
        action,

```

```
    typeof detail === 'string' ? detail.slice(0, 1000) : JSON.stringify(detail || {}).slice(0, 1000),
    req.ip || ''
  ]
};
} catch (e) {
  console.error('[oplog] 写入失败:', e.message);
}
}
```

```
module.exports = { writeOpLog };
```

```
// ===== recycle-admin/server/routes/authRoutes.js =====
```

```
/**
 * 认证路由：登录 / 当前信息 / 修改密码 / 维修后台单点登录
 */
const express = require('express');
const bcrypt = require('bcryptjs');
const crypto = require('crypto');
const https = require('https');
const router = express.Router();
const db = require('../database');
const { signToken, authenticate } = require('../middleware/auth');
const { writeOpLog } = require('../utils/oplog');

/**
 * 拿维修后台的 access_token 去主系统换用户信息（服务端对服务端，走网关）。
 * 主系统校验通过才认可该 token，本系统不持有主系统的 JWT 密钥。
 */
function fetchMainProfile(mainToken) {
  const base = new URL(process.env.MAIN_API_BASE || 'https://nginx/api');
  const host = process.env.MAIN_API_HOST || 'zych.net.cn';
  return new Promise((resolve) => {
    const req = https.request(
      {
        protocol: base.protocol,
        hostname: base.hostname,
        port: base.port || 443,
        path: `${base.pathname.replace(/\/$/, '')}/auth/profile`,
        method: 'GET',
        servername: host,
        headers: {
          Authorization: `Bearer ${mainToken}`,
          Host: host,
          Accept: 'application/json'
        }
      },
      // 网关证书签给 zych.net.cn，容器内用服务名 nginx 访问，域名对不上
      rejectUnauthorized: false
    ),
    (res) => {
      let body = '';
      res.on('data', (c) => (body += c));
      res.on('end', () => {
```

```

    if (res.statusCode !== 200) return resolve(null);
    try {
      const json = JSON.parse(body);
      resolve(json && json.code === 200 ? json.data || null : null);
    } catch {
      resolve(null);
    }
  });
}
);
req.on('error', (e) => {
  console.error('[auth/sso] 调用主系统失败:', e.message);
  resolve(null);
});
req.setTimeout(8000, () => req.destroy(new Error('timeout')));
req.end();
});
}

/**
 * POST /api/auth/sso —— 维修后台单点登录
 * 前端两个后台同源部署，维修后台 token 存于 localStorage['token'],
 * 回收后台取到后交给本接口换取回收系统会话。
 * 回收系统账号按 username 匹配；不存在则自动开通（默认 admin 角色，
 * 随机密码，仅可经 SSO 进入），实现“登录一遍、切换免登”。
 */
router.post('/sso', async (req, res) => {
  try {
    const header = req.headers['authorization'] || '';
    const mainToken = (header.startsWith('Bearer ') ? header.slice(7) : '') || (req.body && req.body.token) || '';
    if (!mainToken) {
      return res.status(401).json({ success: false, message: '缺少维修后台登录凭证' });
    }
    const profile = await fetchMainProfile(mainToken);
    if (!profile || !profile.username) {
      return res.status(401).json({ success: false, message: '维修后台登录已过期，请先登录维修后台' });
    }

    let rows = await db.query('SELECT * FROM recycle_admins WHERE username = ?', [String(profile.username)]);
    let admin = rows[0];
    if (!admin) {
      const result = await db.query(
        `INSERT INTO recycle_admins (username, password_hash, name, role, status)
        VALUES (?, ?, ?, 'admin', 1),
        [
          String(profile.username),
          bcrypt.hashSync(crypto.randomBytes(24).toString('hex'), 10),
          profile.real_name || String(profile.username)
        ]
      );
      rows = await db.query('SELECT * FROM recycle_admins WHERE id = ?', [result.insertId]);
      admin = rows[0];
    }
  }
});

```

```
    console.log('[auth/sso] 已为维修后台用户 ${profile.username} 自动开通回收后台账号');
  }
  if (admin.status !== 1) {
    return res.status(403).json({ success: false, message: '账号已禁用, 请联系超级管理员' });
  }

  await db.query('UPDATE recycle_admins SET last_login_at = NOW() WHERE id = ?', [admin.id]);
  const token = signToken(admin);
  req.admin = admin;
  req.ip = req.ip;
  await writeOpLog(req, 'auth', 'sso_login', `管理员 ${admin.username} 从维修后台单点登录`);
  res.json({
    success: true,
    data: {
      token,
      admin: { id: admin.id, username: admin.username, name: admin.name, role: admin.role }
    }
  });
} catch (err) {
  console.error('[auth/sso]', err);
  res.status(500).json({ success: false, message: '服务器错误' });
}
});

router.post('/login', async (req, res) => {
  try {
    const { username, password } = req.body || {};
    if (!username || !password) {
      return res.status(400).json({ success: false, message: '请输入账号和密码' });
    }
    const rows = await db.query(
      'SELECT * FROM recycle_admins WHERE username = ?',
      [String(username).trim()]
    );
    if (rows.length === 0) {
      return res.status(401).json({ success: false, message: '账号或密码错误' });
    }
    const admin = rows[0];
    if (admin.status !== 1) {
      return res.status(403).json({ success: false, message: '账号已禁用, 请联系超级管理员' });
    }
    if (!bcrypt.compareSync(String(password), admin.password_hash)) {
      return res.status(401).json({ success: false, message: '账号或密码错误' });
    }
    await db.query('UPDATE recycle_admins SET last_login_at = NOW() WHERE id = ?', [admin.id]);
    const token = signToken(admin);
    req.admin = admin;
    req.ip = req.ip;
    await writeOpLog(req, 'auth', 'login', `管理员 ${admin.username} 登录`);
    res.json({
      success: true,
      data: {
```

```
    token,
    admin: { id: admin.id, username: admin.username, name: admin.name, role: admin.role }
  }
});
} catch (err) {
  console.error('[auth/login]', err);
  res.status(500).json({ success: false, message: '服务器错误' });
}
});

router.get('/me', authenticate, async (req, res) => {
  const { id, username, name, role, last_login_at } = req.admin;
  res.json({ success: true, data: { id, username, name, role, last_login_at } });
});

router.post('/change-password', authenticate, async (req, res) => {
  try {
    const { oldPassword, newPassword } = req.body || {};
    if (!oldPassword || !newPassword) {
      return res.status(400).json({ success: false, message: '请填写原密码和新密码' });
    }
    if (String(newPassword).length < 6) {
      return res.status(400).json({ success: false, message: '新密码至少6位' });
    }
    const rows = await db.query('SELECT password_hash FROM recycle_admins WHERE id = ?', [req.admin.id]);
    if (!bcrypt.compareSync(String(oldPassword), rows[0].password_hash)) {
      return res.status(400).json({ success: false, message: '原密码错误' });
    }
    await db.query('UPDATE recycle_admins SET password_hash = ? WHERE id = ?', [
      bcrypt.hashSync(String(newPassword), 10), req.admin.id
    ]);
    await writeOpLog(req, 'auth', 'change_password', '修改登录密码');
    res.json({ success: true, message: '密码已修改' });
  } catch (err) {
    console.error('[auth/change-password]', err);
    res.status(500).json({ success: false, message: '服务器错误' });
  }
});

module.exports = router;

// ===== recycle-admin/server/routes/orderRoutes.js =====

/**
 * 回收订单管理
 * 数据来源: 业务库 orders 表 (order_type='recycle') , 与小程序共用同一份数据。
 */
const express = require('express');
const router = express.Router();
const db = require('../database');
const { authenticate, requireWrite } = require('../middleware/auth');
const { writeOpLog } = require('../utils/oplog');
```

```
const STATUS_LABELS = {
  pending: '待确认',
  quoted: '已报价',
  confirmed: '已确认',
  processing: '处理中',
  completed: '已完成',
  review: '待评价',
  cancelled: '已取消'
};

const CONDITION_LABELS = { good: '优', normal: '良', fair: '中', poor: '差', excellent: '优+', mint: '全新' };

/** 组装订单查询字段（兼容不同环境的 orders 可选列） */
async function orderSelect() {
  const cols = await db.orderColumns();
  const fields = [
    'o.id', 'o.order_id', 'o.user_id', 'o.device_type', 'o.problem_description',
    'o.custom_description', 'o.images', 'o.service_type', 'o.brand_id',
    'o.device_model', 'o.device_condition', 'o.estimated_price', 'o.actual_price',
    'o.status', 'o.created_at', 'o.updated_at', 'o.completed_at', 'o.cancel_reason',
    'o.cancel_description', 'o.progress', 'o.quote_price', 'o.quote_description',
    'o.quote_status', 'o.payment_status',
    'u.nickname', 'u.real_name', 'u.phone'
  ];
  if (cols.has('device_type_name')) fields.push('o.device_type_name');
  if (cols.has('admin_unread')) fields.push('o.admin_unread');
  if (cols.has('user_unread')) fields.push('o.user_unread');
  if (cols.has('is_internal')) fields.push('o.is_internal');
  if (cols.has('device_source')) fields.push('o.device_source');
  return fields;
}

/**
 * GET /api/orders
 * 查询参数: keyword(订单号/型号/用户), status, condition, startDate, endDate, page, pageSize
 */
router.get('/', authenticate, async (req, res) => {
  try {
    const {
      keyword = '', status = '', condition = '', startDate = '', endDate = '',
      page = 1, pageSize = 15
    } = req.query;
    const where = ["o.order_type = 'recycle'"];
    const params = [];

    if (keyword) {
      where.push('(o.order_id LIKE ? OR o.device_model LIKE ? OR u.nickname LIKE ? OR u.real_name LIKE ? OR u.phone LIKE ?)');
      const kw = `%${keyword}%`;
      params.push(kw, kw, kw, kw, kw);
    }
    if (status && STATUS_LABELS[status]) {
      where.push('o.status = ?');
    }
  }
}
```

```
    params.push(status);
  }
  if (condition) {
    where.push('o.device_condition = ?');
    params.push(condition);
  }
  if (startDate) {
    where.push('o.created_at >= ?');
    params.push(`${startDate} 00:00:00`);
  }
  if (endDate) {
    where.push('o.created_at <= ?');
    params.push(`${endDate} 23:59:59`);
  }

  const fields = await orderSelect();
  const whereSql = where.join(' AND ');
  const limit = Math.min(parseInt(pageSize) || 15, 100);
  const offset = (Math.max(parseInt(page) || 1, 1) - 1) * limit;

  const listSql = `
    SELECT ${fields.join(', ')}
    FROM orders o
    LEFT JOIN users u ON u.id = o.user_id
    WHERE ${whereSql}
    ORDER BY o.created_at DESC
    LIMIT ? OFFSET ?`;
  const countSql = `
    SELECT COUNT(*) AS total,
      COALESCE(SUM(CASE WHEN o.status IN ('completed') THEN o.actual_price ELSE 0 END), 0) AS completedAmount
    FROM orders o
    LEFT JOIN users u ON u.id = o.user_id
    WHERE ${whereSql}`;

  const [rows, countRows] = await Promise.all([
    db.query(listSql, [...params, limit, offset]),
    db.query(countSql, params)
  ]);

  const list = rows.map((r) => ({
    ...r,
    statusLabel: STATUS_LABELS[r.status] || r.status,
    conditionLabel: CONDITION_LABELS[r.device_condition] || r.device_condition || '',
    images: parseImages(r.images),
    realPrice: r.actual_price != null ? Number(r.actual_price) : null
  }));

  res.json({
    success: true,
    data: {
      list,
      total: Number(countRows[0].total),
    }
  });
```

```

        completedAmount: Number(countRows[0].completedAmount),
        page: Math.max(parseInt(page) || 1, 1),
        pageSize: limit
    }
});
} catch (err) {
    console.error('[orders/list]', err);
    res.status(500).json({ success: false, message: '查询订单失败' });
}
});

/** GET /api/orders/:id 详情 (含用户联系信息与收货地址) */
router.get('/:id', authenticate, async (req, res) => {
    try {
        const id = parseInt(req.params.id);
        if (!id) return res.status(400).json({ success: false, message: '参数错误' });

        const fields = await orderSelect();
        // 地址表列按环境动态拼接 (部分环境缺 detail 等列), 缺失列输出 NULL 保持响应结构稳定
        const addrCols = await db.addressColumns();
        const addrField = (col) => (addrCols.has(col) ? `a.${col}` : `NULL AS ${col}`);
        const rows = await db.query(
            `SELECT ${fields.join(', ')},
                ${addrField('contact_name')}, ${addrField('contact_phone')},
                ${addrField('province')}, ${addrField('city')}, ${addrField('district')}, ${addrField('detail')}
            FROM orders o
            LEFT JOIN users u ON u.id = o.user_id
            LEFT JOIN user_addresses a ON a.id = o.address_id
            WHERE o.id = ? AND o.order_type = 'recycle',
            [id]
        );
        if (rows.length === 0) {
            return res.status(404).json({ success: false, message: '回收订单不存在' });
        }
        const o = rows[0];
        o.statusLabel = STATUS_LABELS[o.status] || o.status;
        o.conditionLabel = CONDITION_LABELS[o.device_condition] || o.device_condition || '';
        o.images = parselImages(o.images);
        o.address = o.contact_name
            ? `${o.province || ''}${o.city || ''}${o.district || ''} ${o.detail || ''} (${o.contact_name} ${o.contact_phone || ''})`
            : '';
        res.json({ success: true, data: o });
    } catch (err) {
        console.error('[orders/detail]', err);
        res.status(500).json({ success: false, message: '查询详情失败' });
    }
});

/**
 * POST /api/orders/:id/quote 提交回收报价
 * body: { price, description }
 * 更新 actual_price/quote_price, 状态置为 quoted, 用户端将看到最新报价。

```

```
*/
router.post('/:id/quote', authenticate, requireWrite, async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const { price, description = '' } = req.body || {};
    const priceNum = Number(price);
    if (!id) return res.status(400).json({ success: false, message: '参数错误' });
    if (!Number.isFinite(priceNum) || priceNum < 0) {
      return res.status(400).json({ success: false, message: '报价金额无效' });
    }

    const rows = await db.query(
      "SELECT id, order_id, device_model, status FROM orders WHERE id = ? AND order_type = 'recycle'",
      [id]
    );
    if (rows.length === 0) {
      return res.status(404).json({ success: false, message: '回收订单不存在' });
    }
    if (rows[0].status === 'completed' || rows[0].status === 'cancelled') {
      return res.status(400).json({ success: false, message: '订单已完结，不能报价' });
    }

    const cols = await db.orderColumns();
    await db.query(
      `UPDATE orders SET
        actual_price = ?, quote_price = ?, quote_description = ?,
        quote_status = 'pending', quote_created_at = NOW()${cols.has('quote_created_by') ? ', quote_created_by = ?' : ''},
        status = 'quoted', updated_at = NOW()
        WHERE id = ?`,
      cols.has('quote_created_by')
        ? [priceNum, priceNum, description, req.admin.id, id]
        : [priceNum, priceNum, description, id]
    );

    await writeOpLog(req, 'order', 'quote', `订单 ${rows[0].order_id} 报价 ¥${priceNum}${description ? ': ' + description : ''}`);
    res.json({ success: true, message: '报价已提交' });
  } catch (err) {
    console.error('[orders/quote]', err);
    res.status(500).json({ success: false, message: '报价失败' });
  }
});

/**
 * POST /api/orders/:id/status 状态流转
 * body: { status, reason? }
 * 回收订单链路: pending → quoted → confirmed → processing → completed; 任意非完结态可 cancelled。
 */
router.post('/:id/status', authenticate, requireWrite, async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const { status, reason = '' } = req.body || {};
    if (!id || !STATUS_LABELS[status]) {
```

```
    return res.status(400).json({ success: false, message: '目标状态无效' });
  }
  const rows = await db.query(
    "SELECT id, order_id, status FROM orders WHERE id = ? AND order_type = 'recycle'",
    [id]
  );
  if (rows.length === 0) {
    return res.status(404).json({ success: false, message: '回收订单不存在' });
  }
  const current = rows[0].status;
  if (current === 'completed' || current === 'cancelled') {
    return res.status(400).json({ success: false, message: '订单已完结，不能变更状态' });
  }
  if (status === current) {
    return res.status(400).json({ success: false, message: '订单已处于该状态' });
  }

  const sets = ['status = ?', 'updated_at = NOW()'];
  const params = [status];
  if (status === 'completed') {
    sets.push('completed_at = NOW()', 'progress = 100');
  }
  if (status === 'cancelled') {
    sets.push('cancelled_at = NOW()', 'cancel_reason = ?', 'cancel_description = ?');
    params.push('admin_cancel', reason || '后台取消');
  }
  params.push(id);
  await db.query('UPDATE orders SET ${sets.join(', ')} WHERE id = ?', params);

  await writeOpLog(req, 'order', 'status', `订单 ${rows[0].order_id}: ${STATUS_LABELS[current]} → ${STATUS_LABELS[status]}${...
  res.json({ success: true, message: `状态已更新为「${STATUS_LABELS[status]}」` });
} catch (err) {
  console.error(['orders/status', err]);
  res.status(500).json({ success: false, message: '状态更新失败' });
}
});

/**
 * POST /api/orders 手动创建回收订单（登记线下回收）
 * body: { userId, deviceModel, condition, estimatedPrice, description, serviceType }
 */
router.post('/', authenticate, requireWrite, async (req, res) => {
  try {
    const {
      userId, deviceModel, condition = 'normal', estimatedPrice = 0,
      description = '', serviceType = 'shop'
    } = req.body || {};
    if (!userId || !deviceModel) {
      return res.status(400).json({ success: false, message: '请选择用户并填写设备型号' });
    }
    const users = await db.query('SELECT id FROM users WHERE id = ?', [parseInt(userId)]);
    if (users.length === 0) {
```

```
    return res.status(400).json({ success: false, message: '用户不存在' });
  }
  const orderNo = `REC${Date.now()}${Math.floor(Math.random() * 90 + 10)}`;
  await db.query(
    `INSERT INTO orders (
      order_id, user_id, order_type, device_type, problem_description, custom_description,
      images, service_type, device_model, device_condition, estimated_price,
      status, created_at, updated_at, progress, priority
    ) VALUES (?, ?, 'recycle', 1, '线下回收登记', ?, NULL, ?, ?, ?, 'pending', NOW(), NOW(), 0, 0),
    [orderNo, parseInt(userId), description, serviceType, String(deviceModel).slice(0, 100), condition, Number(estimatedPrice) || 0]
  );
  await writeOpLog(req, 'order', 'create', `手动创建回收订单 ${orderNo} (${deviceModel})`);
  res.json({ success: true, message: '回收订单已创建', data: { orderNo } });
} catch (err) {
  console.error('orders/create', err);
  res.status(500).json({ success: false, message: '创建订单失败' });
}
});

/** GET /api/orders/export/list 导出筛选结果为 CSV (Excel 可直接打开) */
router.get('/export/list', authenticate, async (req, res) => {
  try {
    const { keyword = '', status = '', startDate = '', endDate = '' } = req.query;
    const where = ["o.order_type = 'recycle'"];
    const params = [];
    if (keyword) {
      where.push('(o.order_id LIKE ? OR o.device_model LIKE ? OR u.phone LIKE ?)');
      const kw = `%${keyword}%`;
      params.push(kw, kw, kw);
    }
    if (status && STATUS_LABELS[status]) {
      where.push('o.status = ?');
      params.push(status);
    }
    if (startDate) { where.push('o.created_at >= ?'); params.push(`${startDate} 00:00:00`); }
    if (endDate) { where.push('o.created_at <= ?'); params.push(`${endDate} 23:59:59`); }

    const rows = await db.query(
      `SELECT o.order_id, o.device_model, o.device_condition, o.estimated_price, o.actual_price,
        o.status, o.created_at, o.completed_at, u.nickname, u.real_name, u.phone
      FROM orders o LEFT JOIN users u ON u.id = o.user_id
      WHERE ${where.join(' AND ')}
      ORDER BY o.created_at DESC LIMIT 5000`,
      params
    );

    const header = ['订单号', '设备型号', '成色', '预估价', '成交价', '状态', '客户', '联系电话', '创建时间', '完成时间'];
    const lines = rows.map((r) => [
      r.order_id, r.device_model || '', CONDITION_LABELS[r.device_condition] || r.device_condition || '',
      r.estimated_price ?? '', r.actual_price ?? '', STATUS_LABELS[r.status] || r.status,
      r.real_name || r.nickname || '', r.phone || '', r.created_at || '', r.completed_at || ''
    ]);
  }
});
```

```
const csv = [header, ...lines]
  .map((cols) => cols.map((c) => `"${String(c ?? '').replace(/"/g, '"')}".join(',')`))
  .join('\r\n');

await writeOpLog(req, 'order', 'export', `导出回收订单 ${rows.length} 条`);
res.setHeader('Content-Type', 'text/csv; charset=utf-8');
res.setHeader('Content-Disposition', `attachment; filename=recycle-orders-${Date.now()}.csv`);
res.send('\uFEFF' + csv);
} catch (err) {
  console.error('[orders/export]', err);
  res.status(500).json({ success: false, message: '导出失败' });
}
});

/** GET /api/orders/meta/options 用户下拉（手动创建订单时选择） */
router.get('/meta/options', authenticate, async (req, res) => {
  try {
    const { keyword = '' } = req.query;
    const params = [];
    let where = '1=1';
    if (keyword) {
      where = '(nickname LIKE ? OR real_name LIKE ? OR phone LIKE ?)';
      const kw = `%${keyword}%`;
      params.push(kw, kw, kw);
    }
    const rows = await db.query(
      `SELECT id, nickname, real_name, phone FROM users WHERE ${where} ORDER BY id DESC LIMIT 30`,
      params
    );
    res.json({ success: true, data: rows });
  } catch (err) {
    console.error('[orders/options]', err);
    res.status(500).json({ success: false, message: '查询用户失败' });
  }
});

function parseImages(value) {
  if (!value) return [];
  if (Array.isArray(value)) return value;
  if (typeof value === 'string') {
    try {
      const parsed = JSON.parse(value);
      return Array.isArray(parsed) ? parsed : [];
    } catch (e) {
      return [];
    }
  }
  return [];
}

module.exports = router;

// ===== recycle-admin/server/routes/catalogRoutes.js =====
```

```
/**
 * 设备配价库管理：分类 → 品牌 → 型号（基准回收价）
 */
const express = require('express');
const router = express.Router();
const db = require('../database');
const { authenticate, requireWrite } = require('../middleware/auth');
const { writeOpLog } = require('../utils/oplog');

/** GET /api/catalog/tree 完整目录树（含每个型号配价） */
router.get('/tree', authenticate, async (req, res) => {
  try {
    const [cats, brands, models] = await Promise.all([
      db.query('SELECT * FROM recycle_categories ORDER BY sort_order, id'),
      db.query('SELECT * FROM recycle_brands ORDER BY sort_order, id'),
      db.query('SELECT id, brand_id, name, specs, base_price, market_price, hot, status, sort_order, updated_at FROM recycle_models ...
    ]);
    const modelMap = new Map();
    for (const m of models) {
      if (!modelMap.has(m.brand_id)) modelMap.set(m.brand_id, []);
      modelMap.get(m.brand_id).push(m);
    }
    const brandMap = new Map();
    for (const b of brands) {
      if (!brandMap.has(b.category_id)) brandMap.set(b.category_id, []);
      brandMap.get(b.category_id).push({ ...b, models: modelMap.get(b.id) || [] });
    }
    const tree = cats.map((c) => ({ ...c, brands: brandMap.get(c.id) || [] }));
    res.json({ success: true, data: tree });
  } catch (err) {
    console.error('[catalog/tree]', err);
    res.status(500).json({ success: false, message: '查询配价库失败' });
  }
});

/** GET /api/catalog/models 分页查询型号（支持按分类/品牌/关键词筛选，用于配价调价表格） */
router.get('/models', authenticate, async (req, res) => {
  try {
    const { categoryId = '', brandId = '', keyword = '', status = '', hot = '', page = 1, pageSize = 20 } = req.query;
    const where = ['1=1'];
    const params = [];
    if (categoryId) { where.push('b.category_id = ?'); params.push(parseInt(categoryId)); }
    if (brandId) { where.push('m.brand_id = ?'); params.push(parseInt(brandId)); }
    if (keyword) { where.push('(m.name LIKE ? OR m.specs LIKE ?)'); params.push(`%${keyword}%`, `%${keyword}%`); }
    if (status !== '') { where.push('m.status = ?'); params.push(parseInt(status)); }
    if (hot !== '') { where.push('m.hot = ?'); params.push(parseInt(hot)); }

    const limit = Math.min(parseInt(pageSize) || 20, 100);
    const offset = (Math.max(parseInt(page) || 1, 1) - 1) * limit;
    const whereSql = where.join(' AND ');

    const [rows, countRows] = await Promise.all([
```

```
db.query(
  `SELECT m.*, b.name AS brand_name, c.name AS category_name, c.id AS category_id
  FROM recycle_models m
  JOIN recycle_brands b ON b.id = m.brand_id
  JOIN recycle_categories c ON c.id = b.category_id
  WHERE ${whereSql}
  ORDER BY c.sort_order, b.sort_order, m.sort_order, m.id
  LIMIT ? OFFSET ?`,
  [...params, limit, offset]
),
db.query(
  `SELECT COUNT(*) AS total
  FROM recycle_models m
  JOIN recycle_brands b ON b.id = m.brand_id
  JOIN recycle_categories c ON c.id = b.category_id
  WHERE ${whereSql}`,
  params
)
});
res.json({
  success: true,
  data: { list: rows, total: Number(countRows[0].total), page: Math.max(parseInt(page) || 1, 1), pageSize: limit }
});
} catch (err) {
  console.error('[catalog/models]', err);
  res.status(500).json({ success: false, message: '查询型号失败' });
}
});

// GET /api/catalog/export/models —— 配价库全量导出 CSV (Excel 可直接打开)
// 鉴权支持 ?token=, 供前端新窗口直接下载
router.get('/export/models', authenticate, async (req, res) => {
  try {
    const rows = await db.query(
      `SELECT c.name AS category, b.name AS brand, m.name AS model, m.specs,
      m.base_price, m.market_price, m.hot, m.status, m.updated_at
      FROM recycle_models m
      JOIN recycle_brands b ON b.id = m.brand_id
      JOIN recycle_categories c ON c.id = b.category_id
      ORDER BY c.sort_order, b.sort_order, m.sort_order, m.id`
    );
    const esc = (v) => {
      const s = v === null || v === undefined ? '' : String(v);
      return /["\n]/.test(s) ? `${s.replace(/"/g, '"')}"` : s;
    };
    const header = ['分类', '品牌', '型号', '规格说明', '回收基准价', '市场参考价', '热门', '状态', '更新时间'];
    const lines = [header.join(',')];
    for (const r of rows) {
      lines.push(
        [r.category, r.brand, r.model, r.specs, r.base_price, r.market_price, Number(r.hot) ? '是' : '否', Number(r.status) ? '上架' : '下架', r...
        .map(esc)
        .join(',')
      );
    }
  }
});
```

```
    );
  }
  // BOM 保证 Excel 识别 UTF-8 中文
  res.setHeader('Content-Type', 'text/csv; charset=utf-8');
  res.setHeader('Content-Disposition', `attachment; filename="recycle-catalog-${Date.now()}.csv"`);
  res.send("\uFEFF" + lines.join("\r\n"));
} catch (err) {
  console.error('[catalog/export]', err);
  res.status(500).json({ success: false, message: '导出失败' });
}
});

// ===== 分类 =====
router.post('/categories', authenticate, requireWrite, async (req, res) => {
  try {
    const { code, name, icon = '', color = '#5B9E8A', sortOrder = 0 } = req.body || {};
    if (!code || !name) return res.status(400).json({ success: false, message: '请填写分类编码和名称' });
    await db.query(
      'INSERT INTO recycle_categories (code, name, icon, color, sort_order) VALUES (?, ?, ?, ?, ?)',
      [String(code).trim(), name, icon, color, parseInt(sortOrder) || 0]
    );
    await writeOpLog(req, 'catalog', 'category_create', `新增分类「${name}」`);
    res.json({ success: true, message: '分类已创建' });
  } catch (err) {
    if (err && err.code === 'ER_DUP_ENTRY') {
      return res.status(400).json({ success: false, message: '分类编码已存在' });
    }
    console.error('[catalog/createCategory]', err);
    res.status(500).json({ success: false, message: '创建分类失败' });
  }
});

router.put('/categories/:id', authenticate, requireWrite, async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const { name, icon, color, sortOrder, status } = req.body || {};
    const sets = [];
    const params = [];
    if (name !== null) { sets.push('name = ?'); params.push(name); }
    if (icon !== null) { sets.push('icon = ?'); params.push(icon); }
    if (color !== null) { sets.push('color = ?'); params.push(color); }
    if (sortOrder !== null) { sets.push('sort_order = ?'); params.push(parseInt(sortOrder) || 0); }
    if (status !== null) { sets.push('status = ?'); params.push(parseInt(status) ? 1 : 0); }
    if (sets.length === 0) return res.status(400).json({ success: false, message: '无更新内容' });
    params.push(id);
    await db.query('UPDATE recycle_categories SET ${sets.join(', ')} WHERE id = ?', params);
    await writeOpLog(req, 'catalog', 'category_update', `修改分类 #${id}`);
    res.json({ success: true, message: '分类已更新' });
  } catch (err) {
    console.error('[catalog/updateCategory]', err);
    res.status(500).json({ success: false, message: '更新分类失败' });
  }
});
```

```
});

router.delete('/categories/:id', authenticate, requireWrite, async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const used = await db.query(
      'SELECT (SELECT COUNT(*) FROM recycle_brands WHERE category_id = ?) AS brands', [id]
    );
    if (Number(used[0].brands) > 0) {
      return res.status(400).json({ success: false, message: '分类下仍有品牌，请先清空品牌' });
    }
    await db.query('DELETE FROM recycle_categories WHERE id = ?', [id]);
    await writeOpLog(req, 'catalog', 'category_delete', `删除分类 #${id}`);
    res.json({ success: true, message: '分类已删除' });
  } catch (err) {
    console.error('[catalog/deleteCategory]', err);
    res.status(500).json({ success: false, message: '删除分类失败' });
  }
});

// ===== 品牌 =====
router.post('/brands', authenticate, requireWrite, async (req, res) => {
  try {
    const { categoryId, name, logoText = '', logoColor = '#666666', sortOrder = 0 } = req.body || {};
    if (!categoryId || !name) return res.status(400).json({ success: false, message: '请选择分类并填写品牌名称' });
    await db.query(
      'INSERT INTO recycle_brands (category_id, name, logo_text, logo_color, sort_order) VALUES (?, ?, ?, ?, ?)',
      [parseInt(categoryId), name, logoText || name.substring(0, 2), logoColor, parseInt(sortOrder) || 0]
    );
    await writeOpLog(req, 'catalog', 'brand_create', `新增品牌「${name}」`);
    res.json({ success: true, message: '品牌已创建' });
  } catch (err) {
    console.error('[catalog/createBrand]', err);
    res.status(500).json({ success: false, message: '创建品牌失败' });
  }
});

router.put('/brands/:id', authenticate, requireWrite, async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const { name, logoText, logoColor, sortOrder, status } = req.body || {};
    const sets = [];
    const params = [];
    if (name !== null) { sets.push('name = ?'); params.push(name); }
    if (logoText !== null) { sets.push('logo_text = ?'); params.push(logoText); }
    if (logoColor !== null) { sets.push('logo_color = ?'); params.push(logoColor); }
    if (sortOrder !== null) { sets.push('sort_order = ?'); params.push(parseInt(sortOrder) || 0); }
    if (status !== null) { sets.push('status = ?'); params.push(parseInt(status) ? 1 : 0); }
    if (sets.length === 0) return res.status(400).json({ success: false, message: '无更新内容' });
    params.push(id);
    await db.query('UPDATE recycle_brands SET ${sets.join(', ')} WHERE id = ?', params);
    await writeOpLog(req, 'catalog', 'brand_update', `修改品牌 #${id}`);
  }
});
```

```
res.json({ success: true, message: '品牌已更新' });
} catch (err) {
  console.error('[catalog/updateBrand]', err);
  res.status(500).json({ success: false, message: '更新品牌失败' });
}
});

router.delete('/brands/:id', authenticate, requireWrite, async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const used = await db.query('SELECT COUNT(*) AS c FROM recycle_models WHERE brand_id = ?', [id]);
    if (Number(used[0].c) > 0) {
      return res.status(400).json({ success: false, message: '品牌下仍有型号，请先清空型号' });
    }
    await db.query('DELETE FROM recycle_brands WHERE id = ?', [id]);
    await writeOpLog(req, 'catalog', 'brand_delete', `删除品牌 #${id}`);
    res.json({ success: true, message: '品牌已删除' });
  } catch (err) {
    console.error('[catalog/deleteBrand]', err);
    res.status(500).json({ success: false, message: '删除品牌失败' });
  }
});

// ===== 型号（配价） =====
router.post('/models', authenticate, requireWrite, async (req, res) => {
  try {
    const { brandId, name, specs = '', basePrice = 0, marketPrice = null, hot = 0 } = req.body || {};
    if (!brandId || !name) return res.status(400).json({ success: false, message: '请选择品牌并填写型号名称' });
    const result = await db.query(
      `INSERT INTO recycle_models (brand_id, name, specs, base_price, market_price, hot, updated_by)
      VALUES (?, ?, ?, ?, ?, ?, ?)`,
      [parseInt(brandId), name, specs, Number(basePrice) || 0, marketPrice != null && marketPrice != '' ? Number(marketPrice) : null, hot, req.admin.id]
    );
    await db.query(
      `INSERT INTO recycle_price_logs (model_id, model_name, old_price, new_price, reason, admin_id, admin_name)
      VALUES (?, ?, NULL, ?, '新增型号', ?, ?)`,
      [result.insertId, name, Number(basePrice) || 0, req.admin.id, req.admin.name || req.admin.username]
    );
    await writeOpLog(req, 'catalog', 'model_create', `新增型号「${name}」基准价 ¥${Number(basePrice) || 0}`);
    res.json({ success: true, message: '型号已创建' });
  } catch (err) {
    console.error('[catalog/createModel]', err);
    res.status(500).json({ success: false, message: '创建型号失败' });
  }
});

/** 单个型号调价（记录调价日志，受 max_price_adjust_percent 约束提示） */
router.put('/models/:id/price', authenticate, requireWrite, async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const { basePrice, reason = '' } = req.body || {};
    const priceNum = Number(basePrice);
```

```
if (!Number.isFinite(priceNum) || priceNum < 0) {
  return res.status(400).json({ success: false, message: '价格无效' });
}
const rows = await db.query('SELECT * FROM recycle_models WHERE id = ?', [id]);
if (rows.length === 0) return res.status(404).json({ success: false, message: '型号不存在' });
const model = rows[0];
const oldPrice = Number(model.base_price);

const settingRows = await db.query(
  "SELECT config_value FROM recycle_settings WHERE config_key = 'max_price_adjust_percent'"
);
const maxPercent = Number(settingRows[0]?.config_value || 20);
const force = !!req.body.force;
if (!force && oldPrice > 0 && Math.abs(priceNum - oldPrice) / oldPrice * 100 > maxPercent) {
  return res.status(400).json({
    success: false,
    needConfirm: true,
    message: `调价幅度超过 ${maxPercent}% (¥${oldPrice} → ¥${priceNum})，请确认是否继续`
  });
}

await db.query('UPDATE recycle_models SET base_price = ?, updated_by = ? WHERE id = ?', [priceNum, req.admin.id, id]);
await db.query(
  `INSERT INTO recycle_price_logs (model_id, model_name, old_price, new_price, reason, admin_id, admin_name)
  VALUES (?,?,?,?,?,?)`,
  [id, model.name, oldPrice, priceNum, reason || '手动调价', req.admin.id, req.admin.name || req.admin.username]
);
await writeOpLog(req, 'catalog', 'price_update', `「${model.name}」 配价 ¥${oldPrice} → ¥${priceNum}${reason ? ': ' + reason : ''}`);
res.json({ success: true, message: `配价已更新为 ¥${priceNum}` });
} catch (err) {
  console.error('[catalog/updatePrice]', err);
  res.status(500).json({ success: false, message: '调价失败' });
}
});

router.put('/models/:id', authenticate, requireWrite, async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const { name, specs, basePrice, marketPrice, hot, status, sortOrder } = req.body || {};
    const sets = [];
    const params = [];
    if (name !== null) { sets.push('name = ?'); params.push(name); }
    if (specs !== null) { sets.push('specs = ?'); params.push(specs); }
    if (basePrice !== null) { sets.push('base_price = ?'); params.push(Number(basePrice) || 0); }
    if (marketPrice !== undefined) { sets.push('market_price = ?'); params.push(marketPrice === '' || marketPrice === null ? null : ...); }
    if (hot !== null) { sets.push('hot = ?'); params.push(hot ? 1 : 0); }
    if (status !== null) { sets.push('status = ?'); params.push(parseInt(status) ? 1 : 0); }
    if (sortOrder !== null) { sets.push('sort_order = ?'); params.push(parseInt(sortOrder) || 0); }
    if (sets.length === 0) return res.status(400).json({ success: false, message: '无更新内容' });
    sets.push('updated_by = ?');
    params.push(req.admin.id);
    params.push(id);
  }
});
```

```
    await db.query(`UPDATE recycle_models SET ${sets.join(', ')} WHERE id = ?`, params);
    await writeOpLog(req, 'catalog', 'model_update', `修改型号 #${id}`);
    res.json({ success: true, message: '型号已更新' });
  } catch (err) {
    console.error('[catalog/updateModel]', err);
    res.status(500).json({ success: false, message: '更新型号失败' });
  }
});

router.delete('/models/:id', authenticate, requireWrite, async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const rows = await db.query('SELECT name FROM recycle_models WHERE id = ?', [id]);
    await db.query('DELETE FROM recycle_models WHERE id = ?', [id]);
    await writeOpLog(req, 'catalog', 'model_delete', `删除型号「${rows[0]?.name || id}」`);
    res.json({ success: true, message: '型号已删除' });
  } catch (err) {
    console.error('[catalog/deleteModel]', err);
    res.status(500).json({ success: false, message: '删除型号失败' });
  }
});

/**
 * POST /api/catalog/models/batch-price 批量调价
 * body: { modelIds: [1,2], mode: 'percent'|'fixed', value: 10, reason }
 * - percent: 在现价基础上上浮/下调百分比 (value 可为负)
 * - fixed: 统一设置为固定值
 */
router.post('/models/batch-price', authenticate, requireWrite, async (req, res) => {
  try {
    const { modelIds = [], mode = 'percent', value = 0, reason = '' } = req.body || {};
    if (!Array.isArray(modelIds) || modelIds.length === 0) {
      return res.status(400).json({ success: false, message: '请选择型号' });
    }
    const numValue = Number(value);
    if (!Number.isFinite(numValue)) {
      return res.status(400).json({ success: false, message: '调整值无效' });
    }
    const placeholders = modelIds.map(() => '?').join(',');
    const models = await db.query(
      `SELECT id, name, base_price FROM recycle_models WHERE id IN (${placeholders})`,
      modelIds.map((x) => parseInt(x))
    );
    if (models.length === 0) return res.status(400).json({ success: false, message: '型号不存在' });

    for (const m of models) {
      const oldPrice = Number(m.base_price);
      let newPrice;
      if (mode === 'fixed') {
        newPrice = Math.max(0, Math.round(numValue));
      } else {
        newPrice = Math.max(0, Math.round(oldPrice * (1 + numValue / 100)));
      }
    }
  }
});
```

```

    }
    await db.query('UPDATE recycle_models SET base_price = ?, updated_by = ? WHERE id = ?', [newPrice, req.admin.id, m.id]);
    await db.query(
      `INSERT INTO recycle_price_logs (model_id, model_name, old_price, new_price, reason, admin_id, admin_name)
      VALUES (?, ?, ?, ?, ?, ?, ?)`,
      [m.id, m.name, oldPrice, newPrice, reason || `批量调价${mode === 'fixed' ? '固定值' : numValue + '%'}`, req.admin.id, r...
    );
  }
  await writeOpLog(req, 'catalog', 'batch_price', `批量调价 ${models.length} 个型号 (${mode}/${numValue}) `);
  res.json({ success: true, message: `已调整 ${models.length} 个型号` });
} catch (err) {
  console.error('[catalog/batchPrice]', err);
  res.status(500).json({ success: false, message: '批量调价失败' });
}
});

```

/** GET /api/catalog/category-factors 各回收品类的估价系数（含型号统计） */

```

router.get('/category-factors', authenticate, async (req, res) => {
  try {
    const [cats, settings, stats] = await Promise.all([
      db.query('SELECT id, code, name, icon FROM recycle_categories ORDER BY sort_order, id'),
      db.query('SELECT config_key, config_value FROM recycle_settings WHERE config_key LIKE 'cat_factor_%'),
      db.query('SELECT b.category_id AS category_id, COUNT(*) AS models, AVG(m.base_price) AS avgPrice
        FROM recycle_models m
        JOIN recycle_brands b ON b.id = m.brand_id
        GROUP BY b.category_id')
    ]);
    const factorMap = Object.fromEntries(settings.map((r) => [r.config_key.replace('cat_factor_', ''), Number(r.config_value)]));
    const statMap = Object.fromEntries(stats.map((r) => [r.category_id, r]));
    res.json({
      success: true,
      data: cats.map((c) => ({
        id: c.id,
        code: c.code,
        name: c.name,
        icon: c.icon || '',
        factor: factorMap[String(c.id)] ?? 1,
        models: Number(statMap[c.id]?.models || 0),
        avgPrice: Math.round(Number(statMap[c.id]?.avgPrice || 0))
      })))
    });
  } catch (err) {
    console.error('[catalog/categoryFactors]', err);
    res.status(500).json({ success: false, message: '查询品类系数失败' });
  }
});

```

/** PUT /api/catalog/category-factors/:id 修改品类估价系数 */

```

router.put('/category-factors/:id', authenticate, requireWrite, async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const factorNum = Number((req.body || {}).factor);

```

```
// ===== 小程序2.0/pages/recycle/recycle.js =====
```

```
// pages/recycle/recycle.js
const { categories } = require('../utils/recycleData.js');
const recycleCatalog = require('../utils/recycleCatalog.js');

const CATEGORY_VISUALS = {
  phone: { icon: '□', scene: '双摄旗舰', badge: '手机', imageUrl: '/pages/recycle/images/phone.webp' },
  computer: { icon: '□', scene: '轻薄性能', badge: '电脑', imageUrl: '/pages/recycle/images/computer.webp' },
  tablet: { icon: '□', scene: '大屏便携', badge: '平板', imageUrl: '/pages/recycle/images/tablet.webp' },
  wearable: { icon: '□', scene: '腕上智能', badge: '穿戴', imageUrl: '/pages/recycle/images/wearable.webp' },
  display: { icon: '□□', scene: '高清大屏', badge: '显示器', imageUrl: '/pages/recycle/images/display.webp' },
  home: { icon: '□', scene: '生活设备', badge: '数码', imageUrl: '/pages/recycle/images/home.webp' },
  camera: { icon: '□', scene: '影像器材', badge: '相机', imageUrl: '/pages/recycle/images/camera.webp' },
  gaming: { icon: '□', scene: '娱乐主机', badge: '游戏', imageUrl: '/pages/recycle/images/gaming.webp' },
  drone: { icon: '□', scene: '航拍飞行', badge: '无人机', imageUrl: '/pages/recycle/images/drone.webp' },
  server: { icon: '□□', scene: '企业算力', badge: '服务器', imageUrl: '/pages/recycle/images/server.webp' },
  network: { icon: '□', scene: '网络互联', badge: '网络设备', imageUrl: '/pages/recycle/images/network.webp' },
  gpu: { icon: '□□', scene: '图形算力', badge: '显卡', imageUrl: '/pages/recycle/images/gpu.webp' }
};

function getCategoryVisual(category) {
  return CATEGORY_VISUALS[category.id] || {
    icon: category.icon || '□',
    scene: category.name.replace('回收', ''),
    badge: '精选'
  };
}

function buildModelVisual(model, category) {
  const visual = getCategoryVisual(category);
  const specsText = model.specs || '';
  const specs = specsText.split('.').map(item => item.trim()).filter(Boolean);
  const familyLabel = (model.name || '')
    .replace(/\s+/g, ' ')
    .split(' ')
    .slice(0, 2)
    .join(' ');
  const heroTone = model.color || category.color || '#5b9e8a';

  return {
    ...model,
    specs,
    specsText,
    visualIcon: visual.icon,
    visualImageUrl: visual.imageUrl,
    visualScene: visual.scene,
    visualBadge: visual.badge,
    familyLabel: familyLabel || model.name,
    heroTone: heroTone
  };
}
```

```
Page({
  data: {
    categories: [],
    currentCategory: null,
    currentCategoryIndex: 0,
    selectedBrandIndex: -1,
    scrollToBrand: "",

    // ===== 云端配价状态 =====
    catalogManaged: false, // 配价是否来自回收后台云端维护
    catalogLoading: true, // 云端配价是否仍在加载

    // ===== 平台比价 =====
    platforms: [], // 回收平台列表（来自回收后台维护）

    // ===== 搜索相关 =====
    searchKeyword: "", // 输入框文本
    searchResults: [], // 搜索结果列表
    searchHistory: [], // 搜索历史
    showSearchPanel: false, // 是否显示搜索面板
    isInternal: false // 是否为公司内部人员（免付款申请）
  },

  onLoad() {
    // 第一步：先用本地内置配价渲染，保证页面秒开、离线可用
    this.applyCategories(categories);

    // 读取搜索历史
    const history = wx.getStorageSync('recycle_search_history') || [];

    // 检测内部人员身份：公司内部人员发起回收免付款申请，无需支付
    const userInfo = wx.getStorageSync('userInfo') || {};

    this.setData({
      searchHistory: history,
      isInternal: userInfo.role === 'internal'
    });

    // 第二步：异步加载回收后台云端维护的最新配价（有更新则覆盖渲染）
    this.loadCloudCatalog();

    // 第三步：加载回收平台比价数据
    this.loadPlatforms();
  },

  /** 将目录数据（本地或云端）处理为页面渲染结构 */
  applyCategories(sourceCategories) {
    // 处理分类数据，只保留必要的字段
    const processedCategories = sourceCategories.map(cat => ({
      id: cat.id,
      name: cat.name,
```

```
icon: cat.icon,
color: cat.color,
visual: getCategoryVisual(cat),
brands: cat.brands.map(brand => ({
  id: brand.id,
  name: brand.name,
  logo: brand.logo,
  logoText: brand.logoText || brand.name.substring(0, 1),
  logoColor: brand.logoColor || '#666',
  models: brand.models.map(model => {
    const visualModel = buildModelVisual({
      id: model.id,
      name: model.name,
      specs: model.specs,
      price: model.basePrice,
      color: model.color,
      bgColor: model.color || '#5b9e8a'
    }, cat);
    return visualModel;
  })
}));
```

```
// 构建全局搜索索引（扁平化所有型号，附带分类/品牌信息）
const searchIndex = [];
sourceCategories.forEach(cat => {
  cat.brands.forEach(brand => {
    brand.models.forEach(model => {
      searchIndex.push({
        modelId: model.id,
        modelName: model.name,
        modelSpecs: model.specs,
        modelPrice: model.basePrice,
        modelColor: model.color,
        modelBgColor: model.color || '#5b9e8a',
        modelVisualIcon: getCategoryVisual(cat).icon,
        modelVisualImageUrl: getCategoryVisual(cat).imageUrl,
        modelVisualScene: getCategoryVisual(cat).scene,
        modelVisualBadge: getCategoryVisual(cat).badge,
        modelFamilyLabel: (model.name || '').split(' ').slice(0, 2).join(' '),
        brandId: brand.id,
        brandName: brand.name,
        brandLogoText: brand.logoText || brand.name.substring(0, 1),
        brandLogoColor: brand.logoColor || '#666',
        categoryId: cat.id,
        categoryName: cat.name,
        categoryIcon: cat.icon
      });
    });
  });
});
```

```
this.setData({
  categories: processedCategories,
  currentCategory: processedCategories[0],
  currentCategoryIndex: 0,
  selectedBrandIndex: -1,
  searchIndex
});
},

/**
 * 加载回收后台云端维护的配价目录
 * 云端有数据时覆盖本地目录（配价以回收后台最新调价为准），失败保持本地
 */
async loadCloudCatalog() {
  try {
    const { managed, categories: cloudCategories } = await recycleCatalog.loadCatalog();
    if (managed && Array.isArray(cloudCategories) && cloudCategories.length > 0) {
      this.applyCategories(cloudCategories);
    }
    this.setData({ catalogManaged: managed, catalogLoading: false });
  } catch (e) {
    this.setData({ catalogManaged: false, catalogLoading: false });
  }
},

/** 加载回收平台比价数据（后台「回收平台管理」维护） */
async loadPlatforms() {
  const platforms = await recycleCatalog.loadPlatforms();
  this.setData({ platforms });
},

/** 平台卡片点击：复制链接并上报跳转统计（小程序无法直接打开外部网址） */
onPlatformTap(e) {
  const platform = e.currentTarget.dataset.platform;
  if (!platform || !platform.url) return;

  recycleCatalog.reportPlatformClick(platform.id);

  wx.showModal({
    title: platform.name,
    content: `小程序暂不支持直接跳转外部网站，已为您复制链接，可粘贴到浏览器打开比价。\\n\\n${platform.url}`,
    confirmText: '复制链接',
    cancelText: '取消',
    success: (res) => {
      if (res.confirm) {
        wx.setClipboardData({
          data: platform.url,
          success: () => {
            wx.showToast({ title: '链接已复制', icon: 'success' });
          }
        });
      }
    }
  })
}
```

```
    }
  });
},

/**
 * 分类图标加载失败时，显示 emoji 兜底
 */
onCategoryImgError(e) {
  const index = e.currentTarget.dataset.index;
  const categories = this.data.categories;
  if (categories && categories[index] && categories[index].visual) {
    const key = `categories[${index}].visual.imageUrl`;
    this.setData({ [key]: '' });
  }
},

onCategoryTap(e) {
  const index = e.currentTarget.dataset.index;
  this.setData({
    currentCategoryId: index,
    currentCategory: this.data.categories[index],
    selectedBrandIndex: -1,
    scrollToBrand: ''
  });
},

onBrandTap(e) {
  const brandId = e.currentTarget.dataset.brandId;
  const index = e.currentTarget.dataset.index;
  this.setData({
    selectedBrandIndex: index,
    scrollToBrand: `brand-${brandId}`
  });
  // 延迟滚动确保视图更新
  setTimeout(() => {
    this.setData({ scrollToBrand: '' });
  }, 300);
},

onModelTap(e) {
  const { model, brand } = e.currentTarget.dataset;
  const category = this.data.currentCategory;

  // 构建跳转参数
  const params = {
    categoryId: category.id,
    categoryName: category.name.replace('回收', ''),
    brandId: brand.id,
    brandName: brand.name,
    brandLogoText: brand.logoText || '',
    brandLogoColor: brand.logoColor || '',
    modelId: model.id,
```

```
modelName: model.name,
modelPrice: model.price,
modelSpecs: model.specsText || model.specs,
modelColor: model.color,
modelBgColor: model.bgColor || "",
modelVisualIcon: model.visualIcon || "",
modelVisualScene: model.visualScene || "",
internal: this.data.isInternal ? 1 : 0
};

wx.showLoading({ title: '正在加载...', mask: true });
wx.navigateTo({
  url: `/pages/recycle-guide/recycle-guide?${this.buildQuery(params)}`
});
},

buildQuery(params) {
  return Object.keys(params)
    .map(key => `${key}=${encodeURIComponent(params[key])}`)
    .join('&');
},

// =====
// 搜索相关方法
// =====

/** 空操作 — 仅用于 catchtap 阻止事件冒泡 */
noop() {},

/** 点击搜索栏 — 打开搜索面板 */
onSearchBarTap() {
  this.setData({ showSearchPanel: true });
},

/** 输入框内容变化 — 防抖搜索 */
onSearchInput(e) {
  // 保留原始值（含空格）同步给输入框，避免输入空格被吃掉
  const rawValue = e.detail.value;
  this.setData({ searchKeyword: rawValue });

  const keyword = rawValue.trim();

  // 清空输入 → 清空结果
  if (!keyword) {
    this.setData({ searchResults: [] });
    return;
  }

  // 防抖
  clearTimeout(this._searchTimer);
  this._searchTimer = setTimeout(() => {
    this.doSearch(keyword);
  }, 500);
}
```

```
    }, 300);
  },

  /**
   * 执行模糊搜索
   * 支持：多关键词 AND 匹配、忽略空格差异 (iphone17 = iphone 17) 、不区分大小写
   */
  doSearch(keyword) {
    const kw = keyword.toLowerCase().trim();
    // 按空格拆分成多个搜索词，全部匹配才算命中
    const tokens = kw.split(/\s+/);
    // 去除所有空格的版本，用于「无空格模糊匹配」
    const kwNoSpace = kw.replace(/\s+/g, '');

    const results = this.data.searchIndex.filter(item => {
      // 拼接全部可搜索字段
      const combined = (
        item.modelName + ' ' +
        item.brandName + ' ' +
        item.modelSpecs + ' ' +
        item.categoryName
      ).toLowerCase();
      // 去空格版本
      const combinedNoSpace = combined.replace(/\s+/g, '');

      // 匹配策略：
      // 1) 多关键词 AND —— 每个词都在 combined 中出现
      // 2) 无空格模糊 —— 去掉空格后整体匹配（输入 iphone17 能命中 iPhone 17）
      const matchAllTokens = tokens.every(t => combined.includes(t));
      const matchNoSpace = combinedNoSpace.includes(kwNoSpace);

      return matchAllTokens || matchNoSpace;
    }).slice(0, 50); // 最多展示50条

    this.setData({ searchResults: results });
  },

  /** 软键盘确认搜索 — 保存历史 */
  onSearchConfirm() {
    const kw = this.data.searchKeyword.trim();
    if (!kw) return;
    this.saveSearchHistory(kw);
  },

  /** 点击搜索结果 */
  onSearchResultTap(e) {
    const item = e.currentTarget.dataset.item;

    // 保存搜索历史
    this.saveSearchHistory(this.data.searchKeyword);

    // 关闭搜索面板
```

```
this.setData({ showSearchPanel: false });

// 显示加载指示器
wx.showLoading({ title: '正在加载...', mask: true });

// 跳转到估价引导页
const params = {
  categoryId: item.categoryId,
  categoryName: item.categoryName.replace('回收', ''),
  brandId: item.brandId,
  brandName: item.brandName,
  brandLogoText: item.brandLogoText || '',
  brandLogoColor: item.brandLogoColor || '',
  modelId: item.modelId,
  modelName: item.modelName,
  modelPrice: item.modelPrice,
  modelSpecs: item.modelSpecs,
  modelColor: item.modelColor,
  modelBgColor: item.modelBgColor || '',
  modelVisualIcon: item.modelVisualIcon || '',
  modelVisualScene: item.modelVisualScene || '',
  internal: this.data.isInternal ? 1 : 0
};

wx.navigateTo({
  url: `/pages/recycle-guide/recycle-guide?${this.buildQuery(params)}`
});

},

/** 点击历史搜索词 */
onHistoryTap(e) {
  const word = e.currentTarget.dataset.word;
  this.setData({ searchKeyword: word });
  this.doSearch(word);
},

/** 保存搜索历史（去重、最多10条） */
saveSearchHistory(keyword) {
  let history = this.data.searchHistory.filter(h => h !== keyword);
  history.unshift(keyword);
  history = history.slice(0, 10);
  this.setData({ searchHistory: history });
  wx.setStorageSync('recycle_search_history', history);
},

/** 清空搜索历史 */
onClearHistory() {
  wx.showModal({
    title: '清空搜索历史',
    content: '确定要清空所有搜索历史吗？',
    success: (res) => {
      if (res.confirm) {
```

```
this.setData({ searchHistory: [] });
wx.removeStorageSync('recycle_search_history');
}
}
});
},

/** 清空输入框 */
onClearInput() {
  this.setData({
    searchKeyword: '',
    searchResults: []
  });
},

/** 关闭搜索面板 */
onSearchCancel() {
  clearTimeout(this._searchTimer);
  this.setData({
    showSearchPanel: false,
    searchKeyword: '',
    searchResults: []
  });
},

});

// ===== 小程序2.0/pages/recycle-guide/recycle-guide.js =====

// pages/recycle-guide/recycle-guide.js
const { guideQuestions, conditionRates } = require('../../utils/recycleData.js');
const { orderApi } = require('../../utils/api.js');
const { loadValuationConfig } = require('../../utils/recycleCatalog.js');

// 二手回收基准系数：即便是“全新未使用”，回收方也以低于全新零售价（约 9 折）回收，
// 避免估价虚高等于 100% 全新零售价，使金额更接近真实二手行情。
// 该系数可由回收综合服务后台（recycle_settings.base_factor）在线调整，未配置时用本地默认值。
const RECYCLE_BASE_FACTOR = 0.9;

Page({
  data: {
    // 产品信息
    product: {
      categoryId: '',
      categoryName: '',
      brandId: '',
      brandName: '',
      brandLogoText: '',
      brandLogoColor: '#666',
      modelId: '',
      modelName: '',
      modelPrice: 0,
      modelSpecs: '',
```

```
    modelColor: '#4f6b84'
  },

  // 问题相关
  questions: [],
  currentQuestionIndex: 0,
  totalQuestions: 0,
  answers: {},
  estimatedPrice: 0,

  // 动画状态
  questionState: 'enter', // enter | exit | visible
  isTransitioning: false,

  // 估价
  isEvaluating: false,
  evaluationComplete: false,
  evaluationText: '',
  priceRange: { min: 0, max: 0 },

  // 地址相关
  selectedAddress: null,
  showAddressPicker: false,
  addressList: [],

  // 提交
  submitting: false,
  showExtraInput: false,
  extraNotes: '',

  // 内部人员免付款申请
  isInternal: false,
  deviceSourceOptions: [
    { key: 'project_return', label: '项目返修' },
    { key: 'warehouse', label: '仓库' },
    { key: 'fixed_asset', label: '固定资产' }
  ],
  deviceSource: ''
},

onLoad(options) {
  const decode = (val) => val ? decodeURIComponent(val) : '';
  const product = {
    categoryId: decode(options.categoryId),
    categoryName: decode(options.categoryName),
    brandId: decode(options.brandId),
    brandName: decode(options.brandName),
    brandLogoText: decode(options.brandLogoText),
    brandLogoColor: decode(options.brandLogoColor),
    modelId: decode(options.modelId),
    modelName: decode(options.modelName),
    modelPrice: parseInt(options.modelPrice) || 0,
```

```
    modelSpecs: decode(options.modelSpecs),
    modelColor: decode(options.modelColor)
  });

  const questions = guideQuestions.map((q, i) => ({
    ...q,
    stepNumber: i + 1
  }));

  // 内部人员身份判定：入口参数 internal=1 或当前登录用户 role=internal
  const guideUserInfo = wx.getStorageSync('userInfo') || {};
  const guidelsInternal = (options && options.internal === '1') || guideUserInfo.role === 'internal';

  this.setData({
    product,
    questions,
    totalQuestions: questions.length,
    estimatedPrice: product.modelPrice,
    baseFactor: RECYCLE_BASE_FACTOR,
    isInternal: guidelsInternal
  });

  // 异步加载回收后台维护的估价配置（基准系数/成色系数），有更新则覆盖本地默认
  this.loadValuationSettings();

  // 入场动画
  setTimeout(() => {
    this.setData({ questionState: 'visible' });
  }, 300);
},

/**
 * 加载回收后台估价配置：
 * - base_factor: 二手回收基准系数
 * - factors: 各评估因子选项系数（按 value 匹配覆盖本地 guideQuestions）
 */
async loadValuationSettings() {
  const config = await loadValuationConfig();
  if (!config.managed) return;

  const updates = { baseFactor: config.baseFactor };

  if (config.factors && Object.keys(config.factors).length > 0) {
    updates.questions = this.data.questions.map((q) => {
      const remoteOptions = config.factors[q.id];
      if (!Array.isArray(remoteOptions)) return q;
      return {
        ...q,
        options: (q.options || []).map((opt) => {
          const remote = remoteOptions.find((r) => r.value === opt.value);
          return remote ? { ...opt, rate: remote.rate } : opt;
        })
      };
    });
  }
}
```

```
    };
  });
}

this.setData(updates);
},

/**
 * 内部人员选择设备来源（项目返修 / 仓库 / 固定资产）
 */
selectDeviceSource(e) {
  const key = e.currentTarget.dataset.key;
  this.setData({ deviceSource: key });
},

onShow() {
  this.loadAddresses().then(() => {
    wx.hideLoading();
  }).catch(() => {
    wx.hideLoading();
  });
  setTimeout(() => { wx.hideLoading(); }, 3000);

  if (this._needRefreshAddress) {
    this._needRefreshAddress = false;
    this.loadAddresses();
  }
},

// ===== 问题流程 =====

onOptionSelect(e) {
  if (this.data.isTransitioning) return;

  const { value, label, rate, desc } = e.currentTarget.dataset;
  const { currentQuestionIndex, answers, product, questions } = this.data;
  const q = questions[currentQuestionIndex];

  // 记录回答
  answers[q.id] = { value, label, rate: rate || 1 };

  // 计算实时估价（含二手回收基准系数，金额更贴近真实行情）
  let quickEstimate = product.modelPrice;
  Object.keys(answers).forEach(key => {
    if (answers[key] && answers[key].rate) {
      quickEstimate *= answers[key].rate;
    }
  });
  quickEstimate *= (this.data.baseFactor || RECYCLE_BASE_FACTOR);

  const nextIndex = currentQuestionIndex + 1;
  const isLast = nextIndex >= questions.length;
```

```
this.setData({
  answers,
  estimatedPrice: Math.round(quickEstimate),
  isTransitioning: true,
  questionState: 'exit'
});

// 延迟切换到下一个问题
setTimeout(() => {
  if (isLast) {
    this.startEvaluation();
  } else {
    this.setData({
      currentQuestionIndex: nextIndex,
      questionState: 'enter',
      isTransitioning: false
    });
    setTimeout(() => {
      this.setData({ questionState: 'visible' });
    }, 100);
  }
}, 350);
},

onExtralInputInput(e) {
  this.setData({ extraNotes: e.detail.value });
},

onExtralInputConfirm() {
  const value = (this.data.extraNotes || '').trim();
  if (!value && guideQuestions[this.data.currentQuestionIndex].required) {
    wx.showToast({ title: '请输入说明内容', icon: 'none' });
    return;
  }
}

const q = this.data.questions[this.data.currentQuestionIndex];
const answers = { ...this.data.answers };
answers[q.id] = { value: value || '无', label: value || '无额外问题', rate: 1 };

const nextIndex = this.data.currentQuestionIndex + 1;

this.setData({
  answers,
  currentQuestionIndex: nextIndex,
  showExtralInput: false
});

if (nextIndex >= this.data.questions.length) {
  this.startEvaluation();
}
},
```

```
onSkipExtra() {
  const q = this.data.questions[this.data.currentQuestionIndex];
  const answers = { ...this.data.answers };
  answers[q.id] = { value: '无', label: '跳过', rate: 1 };

  const nextIndex = this.data.currentQuestionIndex + 1;

  this.setData({
    answers,
    currentQuestionIndex: nextIndex,
    showExtraInput: false
  });

  if (nextIndex >= this.data.questions.length) {
    this.startEvaluation();
  }
},

// ===== 估价逻辑 =====

async startEvaluation() {
  this.setData({ isEvaluating: true });

  const localEstimate = this.calculateLocalEstimate();
  const priceRange = this.calculatePriceRange(localEstimate);
  const evaluationText = this.generateEvaluationText(localEstimate, priceRange);

  try {
    const llmResult = await this.callLLMEvaluation();
    if (llmResult && llmResult.price) {
      this.finishEvaluation(llmResult.price, priceRange, llmResult.reason || evaluationText, true);
    } else {
      this.finishEvaluation(localEstimate, priceRange, evaluationText, false);
    }
  } catch (error) {
    console.log('LLM估价失败, 使用本地计算:', error.message);
    this.finishEvaluation(localEstimate, priceRange, evaluationText, false);
  }
},

calculateLocalEstimate() {
  const { product, answers } = this.data;
  let price = product.modelPrice;
  if (answers.condition) price *= answers.condition.rate;
  if (answers.screen) price *= answers.screen.rate;
  if (answers.function) price *= answers.function.rate;
  if (answers.version) price *= answers.version.rate;
  if (answers.accessories) price *= answers.accessories.rate;
  if (answers['repair-history']) price *= answers['repair-history'].rate;
  // 叠加二手回收基准系数, 使估价更接近真实回收行情 (系数可由回收后台在线调整)
  price *= (this.data.baseFactor || RECYCLE_BASE_FACTOR);
}
```

```
    return Math.round(price);
  },

  calculatePriceRange(estimate) {
    // 区间收紧，围绕点估价小幅浮动，避免虚高
    return {
      min: Math.round(estimate * 0.9),
      max: Math.round(estimate * 1.05)
    };
  },

  generateEvaluationText(estimate, range) {
    const { product, answers } = this.data;
    const condition = answers.condition?.label || '未知';
    const screen = answers.screen?.label || '未知';
    const func = answers.function?.label || '未知';
    const version = answers.version?.label || '未知';

    return `根据您的 **${product.brandName} ${product.modelName}** 的评估：

    • 成色：${condition}
    • 屏幕：${screen}
    • 功能：${func}
    • 版本：${version}

    综合以上信息，我为您估算的回收价格为 ¥${estimate} 左右，价格区间在 ¥${range.min} ~ ¥${range.max}。`;
  },

  async callLLMEvaluation() {
    const token = wx.getStorageSync('token') || '';
    const { product, answers } = this.data;

    return new Promise((resolve, reject) => {
      const { getApiBaseCandidates } = require('../utils/runtimeConfig.js');
      const baseCandidates = getApiBaseCandidates();
      const baseUrl = baseCandidates[0];
      if (!baseUrl) { reject(new Error('API地址未配置')); return; }

      wx.request({
        url: `${baseUrl}/recycle/evaluate`,
        method: 'POST',
        data: {
          product: {
            category: product.categoryName,
            brand: product.brandName,
            model: product.modelName,
            basePrice: product.modelPrice,
            specs: product.modelSpecs
          },
          answers: Object.keys(answers).reduce((acc, key) => {
            acc[key] = { value: answers[key].value, label: answers[key].label };
            return acc;
          })
        }
      });
    });
  }
}
```

```
    }, {})  
  },  
  header: {  
    'Content-Type': 'application/json',  
    'Authorization': token ? `Bearer ${token}` : ''  
  },  
  success: (res) => {  
    if (res.statusCode === 200 && res.data.success) {  
      resolve(res.data.data);  
    } else {  
      resolve(null);  
    }  
  },  
  fail: () => resolve(null)  
});  
});  
},
```

```
finishEvaluation(price, priceRange, text, fromLLM) {  
  this.setData({  
    estimatedPrice: price,  
    priceRange,  
    evaluationComplete: true,  
    evaluationText: text,  
    isEvaluating: false,  
    fromLLM  
  });  
},
```

```
// ===== 地址管理 =====
```

```
formatAddressList(rawList) {  
  return (rawList || []).map(addr => {  
    if (addr._formatted) return addr;  
    return {  
      ...addr,  
      _formatted: true,  
      id: addr.id || addr.address_id,  
      name: addr.contact_name || addr.contactName || addr.name || '',  
      phone: addr.contact_phone || addr.contactPhone || addr.phone || '',  
      province: addr.province || '',  
      city: addr.city || '',  
      district: addr.district || '',  
      detail: addr.detail_address || addr.detail || '',  
      postalCode: addr.postal_code || addr.postalCode || '',  
      isDefault: !(addr.is_default || addr.isDefault),  
      tags: addr.tags ? (typeof addr.tags === 'string' ? JSON.parse(addr.tags) : addr.tags) : [],  
      user_id: addr.user_id,  
      created_at: addr.created_at  
    };  
  });  
},
```

```
async loadAddresses() {
  try {
    const { addressApi } = require('../utils/api.js');
    const rawList = await addressApi.getAddressList();
    const addressList = this.formatAddressList(Array.isArray(rawList) ? rawList : (rawList.data || []));
    const defaultAddr = addressList.find(a => a.isDefault) || addressList[0] || null;

    this.setData({
      addressList,
      selectedAddress: defaultAddr,
      showAddressPicker: false
    });
  } catch (error) {
    console.log('加载地址失败:', error);
    this.setData({ addressList: [], selectedAddress: null });
  }
},

openAddressPicker() {
  if (this.data.addressList.length === 0) {
    wx.showToast({ title: '暂无地址, 请先添加', icon: 'none' });
    this.goToAddAddress();
    return;
  }
  this.setData({ showAddressPicker: true });
},

closeAddressPicker() {
  this.setData({ showAddressPicker: false });
},

onSelectAddress(e) {
  const index = e.currentTarget.dataset.index;
  const address = this.data.addressList[index];
  this.setData({
    selectedAddress: address,
    showAddressPicker: false
  });
},

goToAddAddress() {
  wx.navigateTo({
    url: '/pages/address-edit/address-edit?mode=add',
    success: () => {
      const pages = getCurrentPages();
      const currentPage = pages[pages.length - 1];
      if (currentPage) {
        currentPage._needRefreshAddress = true;
      }
    }
  });
}
```

```
},

// ===== 提交订单 =====

async onSubmitOrder() {
  if (this.data.submitting) return;
  if (!this.data.selectedAddress) {
    wx.showToast({ title: '请先选择回收地址', icon: 'none' });
    return;
  }

  if (this.data.isInternal && !this.data.deviceSource) {
    wx.showToast({ title: '请选择设备来源', icon: 'none' });
    return;
  }

  this.setData({ submitting: true });
  wx.showLoading({ title: '提交中...' });

  try {
    const { product, answers, selectedAddress, estimatedPrice, isInternal, deviceSource } = this.data;

    const description = Object.keys(answers).map(key => {
      const a = answers[key];
      const q = guideQuestions.find(gq => gq.id === key);
      return `${q ? q.question : key}: ${a.label}`;
    }).join('; ');

    const orderData = {
      orderType: 'recycle',
      deviceId: product.modelId,
      deviceType: product.categoryName,
      deviceTypeName: product.categoryName,
      brand: product.brandName,
      model: product.modelName,
      deviceCondition: answers.condition?.label || '',
      condition: answers.condition?.label || '',
      estimatedPrice,
      problem: description,
      description: `回收估价单\n产品: ${product.brandName} ${product.modelName}\n型号: ${product.modelName}\n\n评估详情: ...`,
      images: [],
      serviceType: 'shop',
      addressId: selectedAddress.id,
      address: `${selectedAddress.name} ${selectedAddress.phone}\n${selectedAddress.province}${selectedAddress.city}${selectedAddress.city}`,
      isInternal,
      is_internal: isInternal ? 1 : 0,
      paymentStatus: isInternal ? 'waived' : 'unpaid',
      deviceSource: isInternal ? deviceSource : ''
    };

    const sourceLabelMap = {
      project_return: '项目返修',
    };
  }
}
```

```
warehouse: '仓库',
fixed_asset: '固定资产'
};
let confirmed = true;
if (isInternal) {
  confirmed = await new Promise((resolve) => {
    wx.showModal({
      title: '提交内部回收申请',
      content: `设备: ${product.brandName} ${product.modelName}\n来源: ${sourceLabelMap[deviceSource] || deviceSource}...`,
      confirmText: '确认提交',
      confirmColor: '#1d4ed8',
      cancelText: '再看看',
      success: (r) => resolve(!r.confirm)
    });
  });
} else {
  // 金额确认 (按市场基准价对比, 确认回收报价是否合理)
  const marketBase = product.modelPrice || 0;
  const pct = marketBase > 0 ? Math.round((estimatedPrice / marketBase) * 100) : 0;
  const confirmMsg =
    `市场基准价 (全新) : ¥${this.formatPrice(marketBase)}\n` +
    `设备成色: ${answers.condition?.label || '未评估'}\n` +
    `本次回收估价: ¥${this.formatPrice(estimatedPrice)}\n` +
    `(成色折算后约为基准价的 ${pct}%, 属合理区间)`;

  confirmed = await new Promise((resolve) => {
    wx.showModal({
      title: '确认回收金额',
      content: confirmMsg,
      confirmText: '确认提交',
      confirmColor: '#3a7a3a',
      cancelText: '再看看',
      success: (r) => resolve(!r.confirm)
    });
  });
}

if (!confirmed) {
  wx.hideLoading();
  this.setData({ submitting: false });
  return;
}

const res = await orderApi.createOrder(orderData);
wx.hideLoading();

if (res && res.success) {
  if (isInternal) {
    wx.showModal({
      title: '内部回收申请已提交',
      content: '您的免付款回收申请已提交, 等待管理员确认后 will 安排回收。',
      showCancel: false,
    });
  }
}
```

```
      confirmText: '知道了',
      success: () => {
        wx.navigateBack({ delta: 2 });
      }
    });
  } else {
    wx.showToast({ title: '回收订单已提交', icon: 'success' });
    setTimeout(() => {
      wx.navigateBack({ delta: 2 });
    }, 1500);
  }
} else {
  wx.showToast({ title: res?.error || res?.message || '提交失败', icon: 'none' });
}
} catch (error) {
  wx.hideLoading();
  console.error('提交回收订单失败:', error);
  wx.showToast({ title: '提交失败，请重试', icon: 'none' });
}

this.setData({ submitting: false });
},

// ===== 工具方法 =====

onBack() {
  if (this.data.evaluationComplete && !this.data.submitting) {
    wx.navigateBack();
  } else if (this.data.currentQuestionIndex > 0) {
    // 可以在这里添加"确认退出"的逻辑
    wx.showModal({
      title: '退出评估',
      content: '确定要退出吗？已填写的信息将会丢失。',
      success: (res) => {
        if (res.confirm) wx.navigateBack();
      }
    });
  } else {
    wx.navigateBack();
  }
},

onContactSupport() {
  // service 是 tabBar 页面，必须用 switchTab，navigateTo 会静默失败
  wx.switchTab({ url: '/pages/service/service' });
},

formatPrice(price) {
  return price.toString().replace(/\B(?=(\d{3})+(?!\d))/g, ',');
}
});

// ===== 小程序2.0/utls/recycleCatalog.js =====
```

```
// utils/recycleCatalog.js - 回收目录云端加载器
// 配价库由独立的「回收综合服务平台后台」维护，小程序优先使用云端最新配价；
// 云端不可用或未初始化时自动回退到本地 utils/recycleData.js 内置数据，保证离线可用。
const localData = require('./recycleData.js');
const { request } = require('./api.js');

const CATALOG_CACHE_KEY = 'recycle_catalog_cache';
const CATALOG_CACHE_TTL = 10 * 60 * 1000; // 配价缓存10分钟

let inflightCatalog = null;

/**
 * 拉取云端配价目录（分类→品牌→型号），结构与 recycleData.categories 一致
 * @returns {Promise<{managed: boolean, categories: Array}>}
 */
function loadCatalog() {
  if (inflightCatalog) return inflightCatalog;

  inflightCatalog = (async () => {
    // 先用最近一次的缓存，避免每次进入页面都等网络
    try {
      const cached = wx.getStorageSync(CATALOG_CACHE_KEY);
      if (cached && cached.expiredAt > Date.now() && Array.isArray(cached.categories)) {
        return { managed: true, categories: cached.categories };
      }
    } catch (e) { /* 缓存读取失败忽略 */ }

    try {
      const res = await request('/recycle/catalog', 'GET', null, { timeout: 8000, suppressErrorToast: true, resolveOnHttpError: true });
      if (res && res.success && res.managed && Array.isArray(res.data) && res.data.length > 0) {
        // 本地有的可视化信息（分类图片等）按分类编码合并，云端目录保持轻量
        const merged = mergeVisuals(res.data);
        try {
          wx.setStorageSync(CATALOG_CACHE_KEY, { expiredAt: Date.now() + CATALOG_CACHE_TTL, categories: merged });
        } catch (e) { /* 存储失败忽略 */ }
        return { managed: true, categories: merged };
      }
    } catch (e) { /* 网络失败走本地 */ }

    return { managed: false, categories: localData.categories };
  })().finally(() => {
    inflightCatalog = null;
  });

  return inflightCatalog;
}

/** 用本地目录的分类视觉资源（图片/场景文案）补齐云端目录 */
function mergeVisuals(cloudCategories) {
  const localByCode = {};
  for (const cat of localData.categories) localByCode[cat.id] = cat;
```

```
return cloudCategories.map((cat) => {
  const localCat = localByCode[cat.id];
  if (!localCat) return cat;
  const localBrandMap = {};
  for (const b of localCat.brands || []) localBrandMap[b.name] = b;

  return {
    ...cat,
    brands: (cat.brands || []).map((brand) => {
      const localBrand = localBrandMap[brand.name];
      // 云端型号未带 color 时沿用同品牌本地第一个型号的颜色，保持卡片观感
      const fallbackColor = (localBrand && localBrand.models && localBrand.models[0] && localBrand.models[0].color) || cat.color;
      return {
        ...brand,
        logo: localBrand ? localBrand.logo : '□□',
        models: (brand.models || []).map((m) => ({
          ...m,
          color: m.color || fallbackColor
        })))
      };
    })
  };
});

/**
 * 拉取回收平台列表（平台比价区数据源）
 * @returns {Promise<Array>} 平台数组；失败返回 []
 */
async function loadPlatforms() {
  try {
    const res = await request('/recycle/platforms', 'GET', null, { timeout: 8000, suppressErrorToast: true, resolveOnHttpError: true });
    if (res && res.success && Array.isArray(res.data)) {
      return res.data;
    }
  } catch (e) { /* 忽略 */ }
  return [];
}

/**
 * 记录平台跳转点击（后台统计跳转来源）
 */
function reportPlatformClick(platformId) {
  if (!platformId) return;
  request('/recycle/click', 'POST', { platformId }, { suppressErrorToast: true, resolveOnHttpError: true }).catch(() => {});
}

/**
 * 拉取估价配置（后台维护的系数与参数）
 * @returns {Promise<{managed: boolean, baseFactor: number, aiEnabled: boolean, notice: string}>}
 */
```

```
async function loadValuationConfig() {
  const fallback = {
    managed: false,
    baseFactor: 0.9,
    aiEnabled: true,
    notice: ""
  };
  try {
    const res = await request('/recycle/config', 'GET', null, { timeout: 8000, suppressErrorToast: true, resolveOnHttpError: true });
    if (res && res.success && res.managed && res.data) {
      const s = res.data.settings || {};
      return {
        managed: true,
        baseFactor: parseFloat(s.base_factor) > 0 ? parseFloat(s.base_factor) : 0.9,
        aiEnabled: s.ai_evaluate_enabled !== '0',
        notice: s.recycle_notice || "",
        factors: res.data.factors || {}
      };
    }
  } catch (e) { /* 忽略 */ }
  return fallback;
}

module.exports = {
  loadCatalog,
  loadPlatforms,
  reportPlatformClick,
  loadValuationConfig,
  mergeVisuals
};

// ===== 小程序2.0/backend/routes/recycleRoutes.js =====

const express = require('express');
const router = express.Router();
const axios = require('axios');
const dotenv = require('dotenv');
const path = require('path');
const db = require('../database');

dotenv.config({ path: path.join(__dirname, '../.env') });

const DEEPSEEK_API_KEY = process.env.Deepseek_api_key || "";
const DEEPSEEK_API_URL = 'https://api.deepseek.com/v1/chat/completions';

/**
 * 回收公开数据接口（供小程序端读取回收后台维护的配价库/平台/配置）
 * 数据表由独立的「回收综合服务平台后台」（recycle-admin，端口3005）维护，
 * 本服务只读共享同一 MySQL 库；表不存在或未初始化时返回 managed=false，
 * 小程序端自动回退到本地内置数据，保证离线可用。
 */

/** recycle_ 表是否已初始化 */
```

```
async function isRecycleCatalogReady() {
  try {
    const rows = await db.query(
      `SELECT COUNT(*) AS c FROM recycle_categories WHERE status = 1`
    );
    return Number(rows[0]?.c || 0) > 0;
  } catch (e) {
    return false;
  }
}

/**
 * GET /api/recycle/catalog - 配价目录（分类→品牌→型号，仅上架）
 * 返回结构与小程序 utils/recycleData.js 的 categories 完全一致
 */
router.get('/catalog', async (req, res) => {
  try {
    if (!(await isRecycleCatalogReady())) {
      return res.json({ success: true, managed: false, data: null });
    }
    const [cats, brands, models] = await Promise.all([
      db.query('SELECT * FROM recycle_categories WHERE status = 1 ORDER BY sort_order, id'),
      db.query('SELECT * FROM recycle_brands WHERE status = 1 ORDER BY sort_order, id'),
      db.query('SELECT * FROM recycle_models WHERE status = 1 ORDER BY sort_order, id')
    ]);

    const modelMap = new Map();
    for (const m of models) {
      if (!modelMap.has(m.brand_id)) modelMap.set(m.brand_id, []);
      modelMap.get(m.brand_id).push({
        id: `db-${m.id}`,
        name: m.name,
        specs: m.specs || '',
        basePrice: Number(m.base_price) || 0,
        hot: m.hot === 1
      });
    }

    const brandMap = new Map();
    for (const b of brands) {
      if (!brandMap.has(b.category_id)) brandMap.set(b.category_id, []);
      brandMap.get(b.category_id).push({
        id: `db-${b.id}`,
        name: b.name,
        logoText: b.logo_text || b.name.substring(0, 2),
        logoColor: b.logo_color || '#666666',
        models: modelMap.get(b.id) || []
      });
    }

    const categories = cats.map((c) => ({
      id: c.code || `db-${c.id}`,
      name: c.name,

```

```
    icon: c.icon || '□',
    color: c.color || '#5B9E8A',
    brands: brandMap.get(c.id) || []
  }));

  res.json({ success: true, managed: true, data: categories });
} catch (error) {
  console.error(['recycle/catalog', error.message]);
  res.json({ success: true, managed: false, data: null });
}
});

/**
 * GET /api/recycle/platforms - 启用中的回收平台（小程序「平台比价」数据源）
 */
router.get('/platforms', async (req, res) => {
  try {
    const rows = await db.query(
      `SELECT id, name, url, type, logo_text, logo_color, description, service_mode, settlement
      FROM recycle_platforms WHERE status = 1 ORDER BY sort_order, id LIMIT 20`
    );
    const platforms = rows.map((r) => ({
      id: r.id,
      name: r.name,
      url: r.url,
      type: r.type,
      typeLabel: r.type === 'procurement' ? '采购渠道' : r.type === 'compare' ? '比价参考' : '回收平台',
      logoText: r.logo_text || r.name.substring(0, 1),
      logoColor: r.logo_color || '#5B9E8A',
      description: r.description || '',
      serviceMode: r.service_mode || '',
      settlement: r.settlement || ''
    }));
    res.json({ success: true, data: platforms });
  } catch (e) {
    // 表未初始化时返回空列表，小程序端隐藏比价区
    res.json({ success: true, data: [] });
  }
});

/**
 * POST /api/recycle/click - 记录小程序端平台跳转点击
 * body: { platformId } 或 { linkId }
 */
router.post('/click', async (req, res) => {
  try {
    const { platformId, linkId } = req.body || {};
    if (platformId) {
      await db.query(`UPDATE recycle_platforms SET click_count = click_count + 1 WHERE id = ?`, [parseInt(platformId)]);
      await db.query(
        "INSERT INTO recycle_click_logs (target_type, target_id, target_name, source) VALUES ('platform', ?, '', 'mini'",
        [parseInt(platformId)]
      );
    }
  }
});
```

```
    );
  } else if (linkId) {
    await db.query('UPDATE recycle_procurement_links SET click_count = click_count + 1, last_click_at = NOW() WHERE id = ?...');
    await db.query(
      "INSERT INTO recycle_click_logs (target_type, target_id, target_name, source) VALUES ('link', ?, '', 'mini')",
      [parseInt(linkId)]
    );
  } else {
    return res.status(400).json({ success: false, message: '缺少平台/链接ID' });
  }
  res.json({ success: true });
} catch (e) {
  res.json({ success: true });
}
});

/**
 * GET /api/recycle/config - 回收估价配置（系数+参数）
 * 小程序估价页用后台配置实时覆盖本地默认系数；未初始化时返回 managed=false。
 */
router.get('/config', async (req, res) => {
  try {
    const [settings, rates] = await Promise.all([
      db.query('SELECT config_key, config_value FROM recycle_settings'),
      db.query('SELECT factor_key, label, value, rate FROM recycle_condition_rates ORDER BY factor_key, sort_order, id')
    ]);
    const config = {};
    for (const s of settings) {
      config[s.config_key] = s.config_value;
    }
    const factors = {};
    for (const r of rates) {
      if (!factors[r.factor_key]) factors[r.factor_key] = [];
      factors[r.factor_key].push({ label: r.label, value: r.value, rate: Number(r.rate) });
    }
    res.json({ success: true, managed: true, data: { settings: config, factors } });
  } catch (e) {
    res.json({ success: true, managed: false, data: null });
  }
});

/**
 * POST /api/recycle/evaluate - LLM回收估价
 * Body: { product: {...}, answers: {...} }
 */
router.post('/evaluate', async (req, res) => {
  try {
    const { product, answers } = req.body;

    if (!product || !answers) {
      return res.status(400).json({
        success: false,

```

```
    message: '缺少产品信息或评估答案'
  });
}

const estimatedPrice = await evaluateWithLLM(product, answers);

res.json({
  success: true,
  data: {
    price: estimatedPrice.price,
    reason: estimatedPrice.reason,
    confidence: estimatedPrice.confidence
  }
});
} catch (error) {
  console.error('回收估价失败:', error.message);
  // 降级：让前端使用本地计算
  res.status(500).json({
    success: false,
    message: 'AI估价暂时不可用，请使用本地估价'
  });
}
});

/**
 * 使用Deepseek LLM进行回收估价
 */
async function evaluateWithLLM(product, answers) {
  const answersSummary = Object.entries(answers)
    .map(([key, val]) => `-${getQuestionLabel(key)}: ${val.label}`)
    .join('\n');

  const systemPrompt = `你是一个专业的电子产品回收估价专家。你需要根据用户提供的产品信息和设备状况，给出一个合理的回收估价。

  估价时请综合考虑以下因素：
  1. 产品型号和当前市场二手价格
  2. 设备成色和使用痕迹
  3. 屏幕状况
  4. 功能完好程度
  5. 配件齐全程度
  6. 是否有维修史
  7. 版本（国行/港版/国际版均影响价格）

  请严格按照JSON格式返回，不要包含其他内容：
  {
    "price": 数字（预估回收价，单位元，整数），
    "reason": "估价说明文字（200字以内，简洁专业）",
    "confidence": "high" | "medium" | "low"
  }
  `;

  const userPrompt = `请对以下二手${product.category}进行回收估价：`
```

【产品信息】

品牌: \${product.brand}

型号: \${product.model}

最高回收价参考: ¥\${product.basePrice}

规格: \${product.specs || '未知'}

【设备状况评估】

\${answersSummary}

请基于以上信息，给出一个合理的回收估价。请严格以JSON格式返回。`;

// 如果没有配置API key, 跳过LLM调用

if (!DEEPSEEK_API_KEY) {

throw new Error('Deepseek API key not configured');

}

const response = await axios.post(

DEEPSEEK_API_URL,

{

model: 'deepseek-chat',

messages: [

{ role: 'system', content: systemPrompt },

{ role: 'user', content: userPrompt }

],

temperature: 0.3,

max_tokens: 500

},

{

headers: {

'Content-Type': 'application/json',

'Authorization': `Bearer \${DEEPSEEK\_API\_KEY}`

},

timeout: 15000

}

);

const content = response.data?.choices?.[0]?.message?.content || "";

// 解析JSON响应

const jsonMatch = content.match(/\s*{.*}\s*/);

if (!jsonMatch) {

throw new Error('LLM响应格式错误');

}

const result = JSON.parse(jsonMatch[0]);

if (!result.price || typeof result.price !== 'number') {

throw new Error('LLM返回的价格无效');

}

return {

price: Math.round(result.price),

```
    reason: result.reason || `根据市场行情和您提供的设备状况评估, 预估回收价格为 ¥${result.price}。`,
    confidence: result.confidence || 'medium'
  };
}
```

```
function getQuestionLabel(key) {
  const labels = {
    'condition': '设备成色',
    'screen': '屏幕状况',
    'function': '功能状况',
    'version': '设备版本',
    'accessories': '配件状况',
    'repair-history': '维修史',
    'extra': '补充说明'
  };
  return labels[key] || key;
}
```

```
module.exports = router;
```

```
// ===== recycle-admin/web/src/api/request.js =====
```

```
import axios from 'axios'
import { ElMessage } from 'element-plus'
import router from '../router'
```

```
const request = axios.create({
  // 子路径部署时由构建注入 VITE_API_BASE (如 /recycle-admin/api) , 默认同源 /api
  baseURL: import.meta.env.VITE_API_BASE || '/api',
  timeout: 20000
})
```

```
request.interceptors.request.use((config) => {
  const token = localStorage.getItem('recycle_admin_token')
  if (token) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})
```

```
request.interceptors.response.use(
  (response) => response.data,
  (error) => {
    const resp = error.response
    if (resp) {
      if (resp.status === 401) {
        localStorage.removeItem('recycle_admin_token')
        localStorage.removeItem('recycle_admin_info')
        if (router.currentRoute.value.path !== '/login') {
          ElMessage.error('登录已过期, 请重新登录')
          router.push('/login')
        }
      } else {

```

```
const data = resp.data
if (data && data.needConfirm) {
  return Promise.reject({ needConfirm: true, message: data.message })
}
ErrorMessage.error((data && data.message) || `请求失败 (${resp.status})`)
}
} else {
  ErrorMessage.error('网络异常, 请检查回收后台服务是否在线')
}
return Promise.reject(error)
}
)
```

export default request

// ===== recycle-admin/web/src/api/index.js =====

import request from './request'

// ===== 认证 =====

```
export const login = (data) => request.post('/auth/login', data)
export const ssoLogin = (mainToken) => request.post('/auth/sso', { token: mainToken })
export const getMe = () => request.get('/auth/me')
export const changePassword = (data) => request.post('/auth/change-password', data)
```

// ===== 估价系数（快速估价用，详见下方系统设置区块的 getConditionRates） =====

// ===== 回收订单 =====

```
export const getOrders = (params) => request.get('/orders', { params })
export const getOrderDetail = (id) => request.get(`/orders/${id}`)
export const quoteOrder = (id, data) => request.post(`/orders/${id}/quote`, data)
export const updateOrderStatus = (id, data) => request.post(`/orders/${id}/status`, data)
export const createOrder = (data) => request.post('/orders', data)
export const getUserOptions = (keyword) => request.get('/orders/meta/options', { params: { keyword } })
export const exportOrderUrl = (params) => `/api/orders/export/list?${new URLSearchParams(params).toString()}`
```

// ===== 配价库 =====

```
export const getCatalogTree = () => request.get('/catalog/tree')
export const getCategoryFactors = () => request.get('/catalog/category-factors')
export const updateCategoryFactor = (id, factor) => request.put(`/catalog/category-factors/${id}`, { factor })
export const getModels = (params) => request.get('/catalog/models', { params })
export const createCategory = (data) => request.post('/catalog/categories', data)
export const updateCategory = (id, data) => request.put(`/catalog/categories/${id}`, data)
export const deleteCategory = (id) => request.delete(`/catalog/categories/${id}`)
export const createBrand = (data) => request.post('/catalog/brands', data)
export const updateBrand = (id, data) => request.put(`/catalog/brands/${id}`, data)
export const deleteBrand = (id) => request.delete(`/catalog/brands/${id}`)
export const createModel = (data) => request.post('/catalog/models', data)
export const updateModel = (id, data) => request.put(`/catalog/models/${id}`, data)
export const updateModelPrice = (id, data) => request.put(`/catalog/models/${id}/price`, data)
export const deleteModel = (id) => request.delete(`/catalog/models/${id}`)
export const batchPrice = (data) => request.post('/catalog/models/batch-price', data)
export const getPriceLogs = (params) => request.get('/catalog/price-logs', { params })
```